

[illegible]

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

This KICAD schematic illustrates the CPU Address Bus Buffering and Nybble Decoding logic. It is organized into several functional blocks:

- CPU Address Bus Buffering:** This block shows the buffering of CPU address pins (CPU_A0_Pin to CPU_A13_Pin) and the M2 clock signal (M2_Pin). The CPU address pins are buffered and then decoded into various CPU address signals (e.g., /CPU_A0{H4}, +CPU_A0{H3}). The M2 clock signal is buffered and then decoded into various M2 clock signals (e.g., /M2_CLK_BUF1{BM21}, +M2_CLK_BUF1{BM21}).
- M2 Clock Buffering:** This block shows the buffering of the M2 clock signal (M2_Pin) and the M2 clock buffer signals (M2_CLK_BUF1, M2_CLK_BUF2, M2_CLK_BUF3). The M2 clock signal is buffered and then decoded into various M2 clock signals (e.g., /M2_CLK_BUF1{BM21}, +M2_CLK_BUF1{BM21}).
- CPU Address Bus Nybble Decoding:** This block shows the decoding of CPU address bus nybbles (CPU_A0{H4} to CPU_A13{H28}) into various CPU address signals (e.g., /CPU_0xx0x{J41}, +CPU_0xx0x{J42}).
- /ROMSEL, Register Access:** This block shows the decoding of the /ROMSEL pin and the M2 clock buffer signals (M2_CLK_BUF3{H34}) into various register access signals (e.g., /Reg_Read{K5}, +Reg_Read{K3}).
- CPU R/W, Register Read/Write:** This block shows the decoding of the CPU_R/W_Pin and the M2 clock buffer signals (M2_CLK_BUF3{H34}) into various register read/write signals (e.g., /Reg_Read{K5}, +Reg_Read{K3}).

The schematic uses a variety of logic components, including buffers, inverters, NAND gates, and decoders, to implement the address bus buffering and nybble decoding logic. The components are labeled with their respective pin numbers and signal names.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

This section shows the buffering of CPU address pins. Pins CPU_A0_Pin through CPU_A13_Pin are connected to a series of inverters and buffers. The outputs are labeled as /CPU_A0{H4}, /CPU_A1{H6}, /CPU_A2{H8}, /CPU_A3{H10}, /CPU_A4{H12}, /CPU_A5{H14}, /CPU_A6{H16}, /CPU_A7{H18}, /CPU_A8{H20}, /CPU_A9{H22}, /CPU_A10{H24}, /CPU_A11{H26}, /CPU_A12{H28}, /CPU_A13{H30}, /CPU_A13{E10}, /CPU_A13{D7}, /CPU_A13{D8}, /CPU_A13{F1}, /CPU_A13{A39}, /CPU_A14{H32}, /CPU_A14{H31}, /CPU_A14{E9}, /CPU_A14{D5}, /CPU_A14{D6}, /CPU_A14{F2}, and /CPU_A14{A40}.

M2 Clock Buffering

This section shows the buffering of the M2 clock signal. The M2_Pin is connected to a series of buffers and inverters. The outputs are labeled as M2_CLK_BUF1{BM21}, M2_CLK_BUF2{B109}, M2_CLK_BUF2{OSC1}, M2_CLK_BUF3{H34}, M2_CLK_BUF3{E12}, M2_CLK_BUF3{E14}, M2_CLK_BUF3{E15}, and M2_CLK_BUF3{A38}.

CPU Address Bus Nybble Decoding

This section shows the decoding of CPU address bus nybbles. It uses a series of NAND gates and inverters to decode the address bus into various nybble signals. The outputs are labeled as /CPU_2xxx{J53}, /CPU_2xxx{J54}, /CPU_4xxx{J51}, /CPU_4xxx{J52}, /CPU_5xxx{J55}, /CPU_5xxx{K27}, /CPU_5xxx{K39}, /CPU_5xxx{K47}, /CPU_5xxx{K59}, /CPU_5xxx{K70}, /CPU_5xxx{K84}, /CPU_x0xx{J49}, /CPU_x0xx{J50}, /CPU_x0xx{K88}, /CPU_x1xx{J47}, /CPU_x1xx{J48}, /CPU_x1xx{K34}, /CPU_x1xx{K44}, /CPU_x1xx{K53}, /CPU_x1xx{K82}, /CPU_x2xx{J45}, /CPU_x2xx{J46}, /CPU_xCxx-xFxx{J43}, and /CPU_xCxx-xFxx{J44}.

CPU R/W, Register Read/Write

This section shows the CPU Read/Write signal and its connection to the register read/write signals. The CPU_R/W_Pin is connected to a series of buffers and inverters. The outputs are labeled as /Reg_Read{K5}, /Reg_Read{K2}, /Reg_Read{K3}, /Reg_Read{L115}, /Reg_Write{K6}, /Reg_Write{BK88}, /Reg_Write{K1}, /Reg_Write{L31}, /Reg_Write{L43}, /Reg_Write{L57}, /Reg_Write{L72}, /Reg_Write{L85}, /Reg_Write{L96}, and /Reg_Write{L111}.

/ROMSEL, Register Access

This section shows the /ROMSEL signal and its connection to the register access signals. The /ROMSEL_Pin is connected to a series of buffers and inverters. The outputs are labeled as /ROMSEL{E23,E24}, /Reg_Access{H36}, /Reg_Access{H37}, /Reg_Access{K18}, /Reg_Access{K30}, /Reg_Access{K36}, /Reg_Access{K48}, /Reg_Access{K63}, /Reg_Access{K74}, /Reg_Access{K98}, and /Reg_Access{H2}.

Notes:

- The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point.
- "Reg_Access" is included in downstream logic to specify which 32kB region.
- Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger **Date:** **Rev:**

KiCad E.D.A. 9.0.7 **Id:** 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

This KICAD schematic diagram illustrates the logic for CPU Address Bus Buffering and Nybble Decoding. The diagram is organized into several functional blocks:

- CPU Address Bus Buffering:** This block shows the buffering of CPU address pins (CPU_A0_Pin to CPU_A13_Pin) through inverters and buffers to generate internal CPU address signals (e.g., /CPU_A0{H4}, +CPU_A0{H3}).
- M2 Clock Buffering:** This block shows the buffering of the M2 clock pin (M2_Pin) through inverters and buffers to generate internal M2 clock signals (e.g., /M2_CLK_BUF1{BM21}, +M2_CLK_BUF1{BM21}).
- CPU Address Bus Nybble Decoding:** This block shows the decoding of CPU address nybbles (e.g., /CPU_A7{H18}, /CPU_A6{H16}) through NAND gates and inverters to generate internal CPU address signals (e.g., /CPU_2xxx{J53}, +CPU_2xxx{J54}).
- /ROMSEL, Register Access:** This block shows the logic for register access, including the /ROMSEL pin and the CPU_R/W_Pin, which are used to generate internal signals for register access (e.g., /Reg_Read{K5}, +Reg_Read{K3}).

The diagram includes a note: "The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point." and "Reg_Access" is included in downstream logic to specify which 32kB region.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

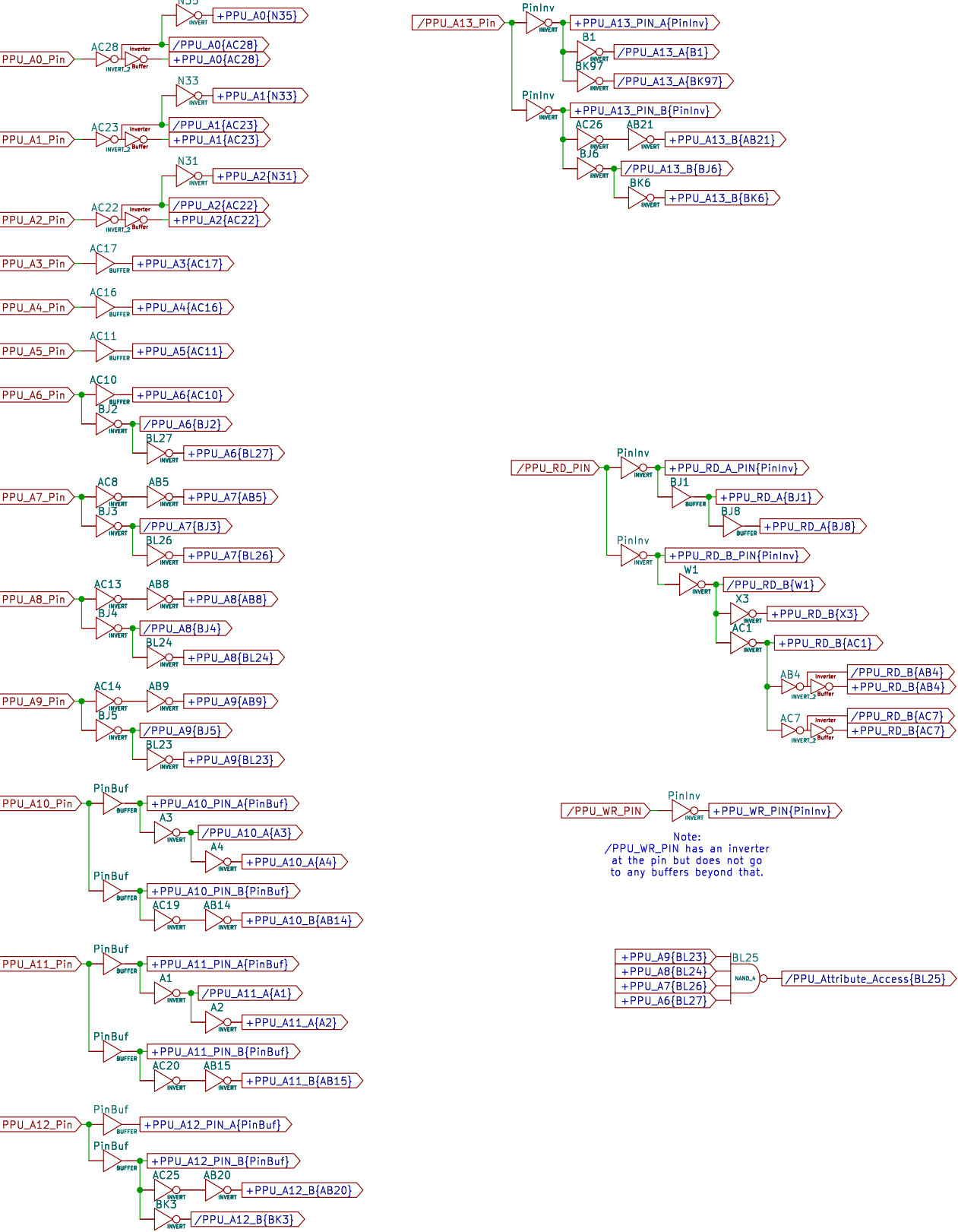
Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

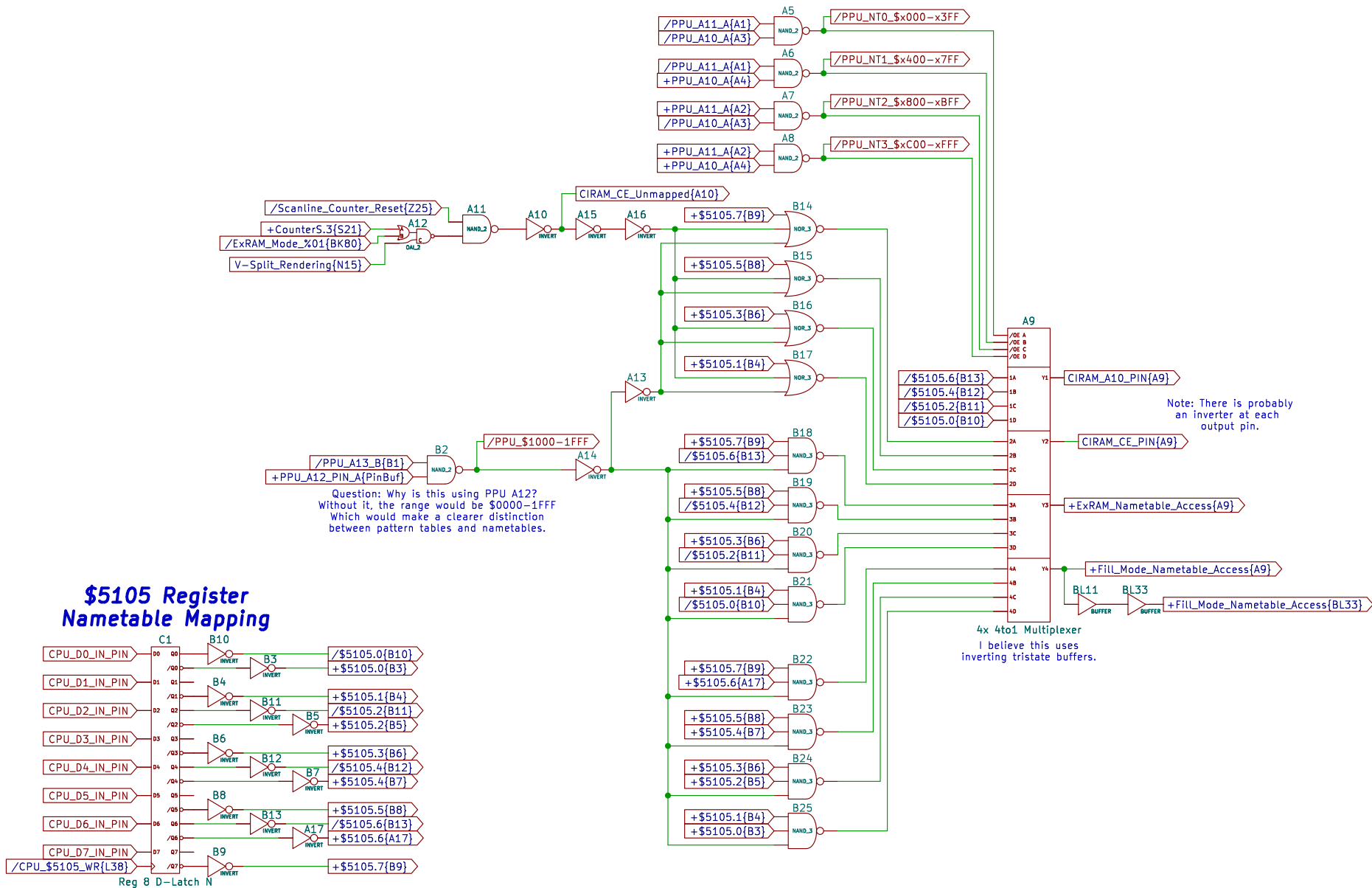
Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

PPU Address Bus Buffering



Note:
/PPU_WR_PIN has an inverter
at the pin but does not go
to any buffers beyond that.



\$5100 Register PRG Mode

Reg 2 DFF-Set

PRG Bank Registers \$5113 Register

Reg 4 D-Latch

\$5114 Register

Reg 8 D-Latch

\$5115 Register

Reg 8 D-Latch

\$5116 Register

Reg 8 D-Latch

\$5117 Register

Reg 8 DFF-Set

Note: \$5117.7 RAM/ROM bit is fully implemented but is somehow being prevented from entering RAM mode when bench testing.

\$5102 Register PRG-RAM Write Protection

Reg 2 DFF-Reset N

\$5103 Register

Reg 2 DFF-Reset N

Note: /Set vs. /Reset inputs are individually used but not easily shown here with a symbol.

PRG-RAM Write Protection

PRG-RAM_A15_Pin

PRG-RAM_A15_Prefbuf

PRG-RAM_A14_Pin

PRG-RAM_A14_Prefbuf

PRG-RAM_A16_Pin

PRG-RAM_A16_Prefbuf

PRG-RAM_A17_Pin

PRG-RAM_A17_Prefbuf

PRG-RAM_A18_Pin

PRG-RAM_A18_Prefbuf

PRG-RAM_A19_Pin

PRG-RAM_A19_Prefbuf

PRG-RAM_A20_Pin

PRG-RAM_A20_Prefbuf

PRG-RAM_A21_Pin

PRG-RAM_A21_Prefbuf

PRG-RAM_A22_Pin

PRG-RAM_A22_Prefbuf

PRG-RAM_A23_Pin

PRG-RAM_A23_Prefbuf

PRG-RAM_A24_Pin

PRG-RAM_A24_Prefbuf

PRG-RAM_A25_Pin

PRG-RAM_A25_Prefbuf

PRG-RAM_A26_Pin

PRG-RAM_A26_Prefbuf

PRG-RAM_A27_Pin

PRG-RAM_A27_Prefbuf

PRG-RAM_A28_Pin

PRG-RAM_A28_Prefbuf

PRG-RAM_A29_Pin

PRG-RAM_A29_Prefbuf

PRG-RAM_A30_Pin

PRG-RAM_A30_Prefbuf

PRG-RAM_A31_Pin

PRG-RAM_A31_Prefbuf

PRG-RAM_A32_Pin

PRG-RAM_A32_Prefbuf

PRG-RAM_A33_Pin

PRG-RAM_A33_Prefbuf

PRG-RAM_A34_Pin

PRG-RAM_A34_Prefbuf

PRG-RAM_A35_Pin

PRG-RAM_A35_Prefbuf

PRG-RAM_A36_Pin

PRG-RAM_A36_Prefbuf

PRG-RAM_A37_Pin

PRG-RAM_A37_Prefbuf

PRG-RAM_A38_Pin

PRG-RAM_A38_Prefbuf

PRG-RAM_A39_Pin

PRG-RAM_A39_Prefbuf

PRG-RAM_A40_Pin

PRG-RAM_A40_Prefbuf

PRG-RAM_A41_Pin

PRG-RAM_A41_Prefbuf

PRG-RAM_A42_Pin

PRG-RAM_A42_Prefbuf

PRG-RAM_A43_Pin

PRG-RAM_A43_Prefbuf

PRG-RAM_A44_Pin

PRG-RAM_A44_Prefbuf

PRG-RAM_A45_Pin

PRG-RAM_A45_Prefbuf

PRG-RAM_A46_Pin

PRG-RAM_A46_Prefbuf

PRG-RAM_A47_Pin

PRG-RAM_A47_Prefbuf

PRG-RAM_A48_Pin

PRG-RAM_A48_Prefbuf

PRG-RAM_A49_Pin

PRG-RAM_A49_Prefbuf

PRG-RAM_A50_Pin

PRG-RAM_A50_Prefbuf

PRG-RAM_A51_Pin

PRG-RAM_A51_Prefbuf

PRG-RAM_A52_Pin

PRG-RAM_A52_Prefbuf

PRG-RAM_A53_Pin

PRG-RAM_A53_Prefbuf

PRG-RAM_A54_Pin

PRG-RAM_A54_Prefbuf

PRG-RAM_A55_Pin

PRG-RAM_A55_Prefbuf

PRG-RAM_A56_Pin

PRG-RAM_A56_Prefbuf

PRG-RAM_A57_Pin

PRG-RAM_A57_Prefbuf

PRG-RAM_A58_Pin

PRG-RAM_A58_Prefbuf

PRG-RAM_A59_Pin

PRG-RAM_A59_Prefbuf

PRG-RAM_A60_Pin

PRG-RAM_A60_Prefbuf

PRG-RAM_A61_Pin

PRG-RAM_A61_Prefbuf

PRG-RAM_A62_Pin

PRG-RAM_A62_Prefbuf

PRG-RAM_A63_Pin

PRG-RAM_A63_Prefbuf

PRG-RAM_A64_Pin

PRG-RAM_A64_Prefbuf

PRG-RAM_A65_Pin

PRG-RAM_A65_Prefbuf

PRG-RAM_A66_Pin

PRG-RAM_A66_Prefbuf

PRG-RAM_A67_Pin

PRG-RAM_A67_Prefbuf

PRG-RAM_A68_Pin

PRG-RAM_A68_Prefbuf

PRG-RAM_A69_Pin

PRG-RAM_A69_Prefbuf

PRG-RAM_A70_Pin

PRG-RAM_A70_Prefbuf

PRG-RAM_A71_Pin

PRG-RAM_A71_Prefbuf

PRG-RAM_A72_Pin

PRG-RAM_A72_Prefbuf

PRG-RAM_A73_Pin

PRG-RAM_A73_Prefbuf

PRG-RAM_A74_Pin

PRG-RAM_A74_Prefbuf

PRG-RAM_A75_Pin

PRG-RAM_A75_Prefbuf

PRG-RAM_A76_Pin

PRG-RAM_A76_Prefbuf

PRG-RAM_A77_Pin

PRG-RAM_A77_Prefbuf

PRG-RAM_A78_Pin

PRG-RAM_A78_Prefbuf

PRG-RAM_A79_Pin

PRG-RAM_A79_Prefbuf

PRG-RAM_A80_Pin

PRG-RAM_A80_Prefbuf

PRG-RAM_A81_Pin

PRG-RAM_A81_Prefbuf

PRG-RAM_A82_Pin

PRG-RAM_A82_Prefbuf

PRG-RAM_A83_Pin

PRG-RAM_A83_Prefbuf

PRG-RAM_A84_Pin

PRG-RAM_A84_Prefbuf

PRG-RAM_A85_Pin

PRG-RAM_A85_Prefbuf

PRG-RAM_A86_Pin

PRG-RAM_A86_Prefbuf

PRG-RAM_A87_Pin

PRG-RAM_A87_Prefbuf

PRG-RAM_A88_Pin

PRG-RAM_A88_Prefbuf

PRG-RAM_A89_Pin

PRG-RAM_A89_Prefbuf

PRG-RAM_A90_Pin

PRG-RAM_A90_Prefbuf

PRG-RAM_A91_Pin

PRG-RAM_A91_Prefbuf

PRG-RAM_A92_Pin

PRG-RAM_A92_Prefbuf

PRG-RAM_A93_Pin

PRG-RAM_A93_Prefbuf

PRG-RAM_A94_Pin

PRG-RAM_A94_Prefbuf

PRG-RAM_A95_Pin

PRG-RAM_A95_Prefbuf

PRG-RAM_A96_Pin

PRG-RAM_A96_Prefbuf

PRG-RAM_A97_Pin

PRG-RAM_A97_Prefbuf

PRG-RAM_A98_Pin

PRG-RAM_A98_Prefbuf

PRG-RAM_A99_Pin

PRG-RAM_A99_Prefbuf

PRG-RAM_A100_Pin

PRG-RAM_A100_Prefbuf

PRG-RAM_A101_Pin

PRG-RAM_A101_Prefbuf

PRG-RAM_A102_Pin

PRG-RAM_A102_Prefbuf

PRG-RAM_A103_Pin

PRG-RAM_A103_Prefbuf

PRG-RAM_A104_Pin

PRG-RAM_A104_Prefbuf

PRG-RAM_A105_Pin

PRG-RAM_A105_Prefbuf

PRG-RAM_A106_Pin

PRG-RAM_A106_Prefbuf

PRG-RAM_A107_Pin

PRG-RAM_A107_Prefbuf

PRG-RAM_A108_Pin

PRG-RAM_A108_Prefbuf

PRG-RAM_A109_Pin

PRG-RAM_A109_Prefbuf

PRG-RAM_A110_Pin

PRG-RAM_A110_Prefbuf

PRG-RAM_A111_Pin

PRG-RAM_A111_Prefbuf

PRG-RAM_A112_Pin

PRG-RAM_A112_Prefbuf

PRG-RAM_A113_Pin

PRG-RAM_A113_Prefbuf

PRG-RAM_A114_Pin

PRG-RAM_A114_Prefbuf

PRG-RAM_A115_Pin

PRG-RAM_A115_Prefbuf

PRG-RAM_A116_Pin

PRG-RAM_A116_Prefbuf

PRG-RAM_A117_Pin

PRG-RAM_A117_Prefbuf

PRG-RAM_A118_Pin

PRG-RAM_A118_Prefbuf

PRG-RAM_A119_Pin

PRG-RAM_A119_Prefbuf

PRG-RAM_A120_Pin

PRG-RAM_A120_Prefbuf

PRG-RAM_A121_Pin

PRG-RAM_A121_Prefbuf

PRG-RAM_A122_Pin

PRG-RAM_A122_Prefbuf

PRG-RAM_A123_Pin

PRG-RAM_A123_Prefbuf

PRG-RAM_A124_Pin

PRG-RAM_A124_Prefbuf

PRG-RAM_A125_Pin

PRG-RAM_A125_Prefbuf

PRG-RAM_A126_Pin

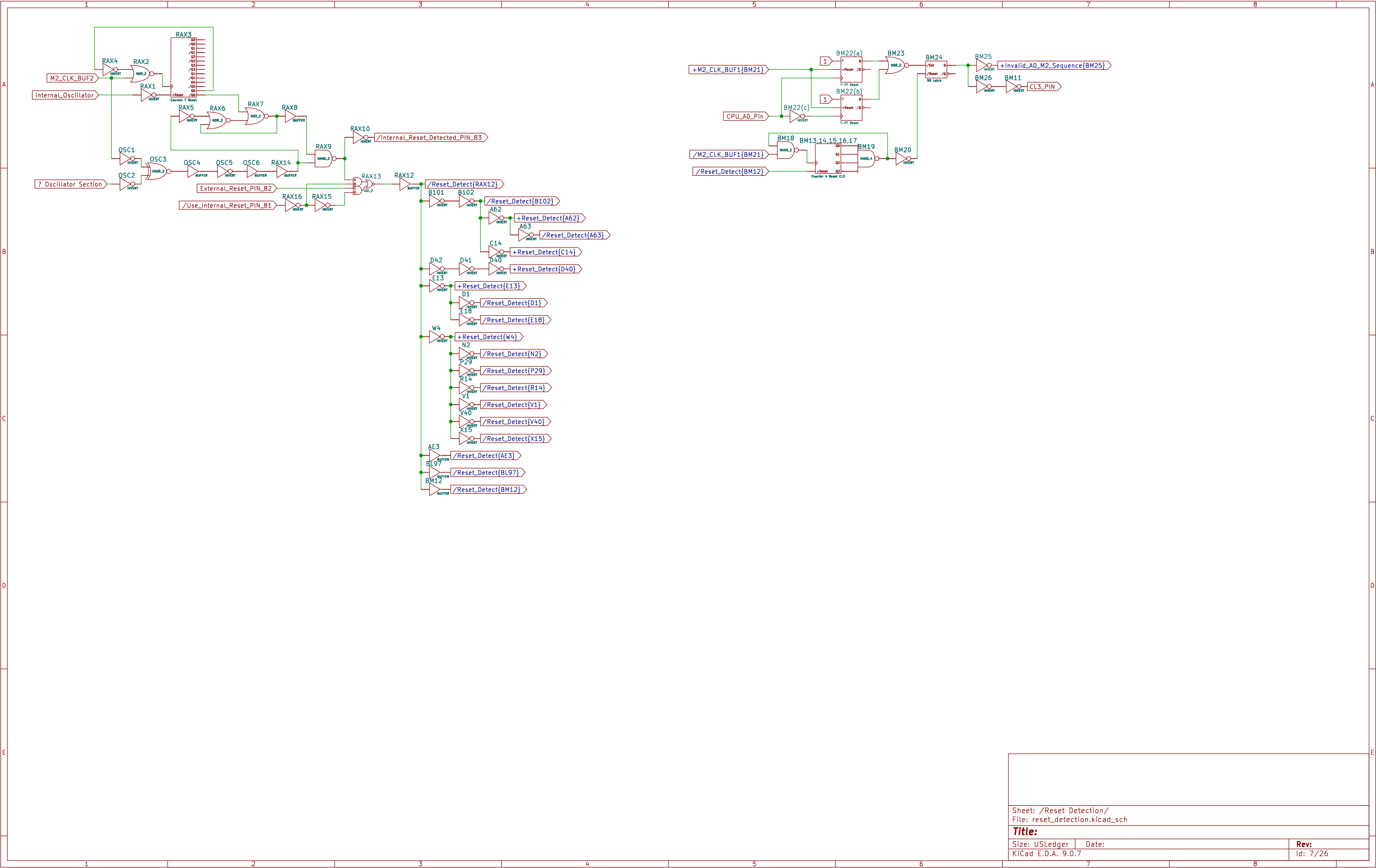
PRG-RAM_A126_Prefbuf

PRG-RAM_A127_Pin

PRG-RAM_A127_P

Sheet: /PRG Banking/
File: prg_banking.kicad_sch

Title:	
Size: USLedger	Date:
KiCad E.D.A. 9.0.7	Rev: Id: 6/26



This diagram illustrates the CPU Data Bus architecture, divided into three main functional blocks: Incoming CPU Data Bus Buffering, Multiplexing Data to be Driven onto the CPU Data Bus, and Decides When to Drive the CPU Data Bus.

Incoming CPU Data Bus Buffering Bonanza

This section shows the buffering of incoming CPU data for addresses 0 through 7. Each address (CPU_IN_PIN) is connected to a series of buffers and inverters to produce both the original data and its complement. For example, CPU_D0_IN_PIN is buffered to produce +CPU_D0{E3}, +CPU_D0{D2}, +CPU_D0{E16}, +CPU_D0{G16}, +CPU_D0{L122}, +CPU_D0{X18}, +CPU_D0{P26}, +CPU_D0{Y4}, +CPU_D0{AE16}, +CPU_D0{AE18}, +CPU_D0{MUL19}, and /CPU_D0{E3}.

Multiplexing Data to be Driven onto the CPU Data Bus

This section shows the multiplexing of data to be driven onto the CPU data bus. It includes ExRAM data (ExRAM_D0_Read{BJ12(a)} through ExRAM_D7_Read{BJ12(h)}), various multiply results (\$5206.0_Read_Multiply_Result_D8{BL78} through \$5205.6_Read_Multiply_Result_D6{BL72}), and status bits (\$5010.7_Read_PCM_IRQ_Pending{BL62}, \$5204.7_Read_Scanline_IRQ_Pending{BL79}). The data is multiplexed using AND gates (BK53 through BK75) and OR gates (BK54 through BK74) to produce the final CPU data bus output (CPU_Dx_OUT_PIN).

Decides When to Drive the CPU Data Bus

This section shows the logic that decides when to drive the CPU data bus. It includes CPU access signals (+CPU_\$5015_Access{L118}, +CPU_\$5204_Access{L88}, +CPU_\$5205_Access{L91}, +CPU_\$5206_Access{L92}), ExRAM access (+CPU_ExRAM_Access{L117}), PPU read/write mode (PPU_RW_Mode_%0x_Access{BL83}), CPU access (+CPU_\$5010_Access{L22}), and register read (/Reg_Read{K2}). These signals are combined using AND gates (BP1 through BP4) and OR gates (BK52 through BK51) to produce the final CPU data bus output (CPU_Dx_Direction{BK45}).

Sheet: /CPU Data Bus/
File: cpu_data_bus.kicad_sch

Title:

Size: USLedger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 8/26

This diagram illustrates the CPU Data Bus architecture, divided into three main functional blocks: Incoming CPU Data Bus Buffering, Multiplexing Data to be Driven onto the CPU Data Bus, and Decides When to Drive the CPU Data Bus.

Incoming CPU Data Bus Buffering Bonanza

This section shows the buffering of incoming CPU data for addresses 0 through 7. Each address (CPU_IN_PIN) is connected to a series of buffers and inverters to generate the corresponding CPU data signals (CPU_OUT_PIN).

- CPU_0:** CPU_0_IN_PIN is buffered and inverted to produce CPU_0_OUT_PIN signals for addresses 0 through 19.
- CPU_1:** CPU_1_IN_PIN is buffered and inverted to produce CPU_1_OUT_PIN signals for addresses 0 through 19.
- CPU_2:** CPU_2_IN_PIN is buffered and inverted to produce CPU_2_OUT_PIN signals for addresses 0 through 19.
- CPU_3:** CPU_3_IN_PIN is buffered and inverted to produce CPU_3_OUT_PIN signals for addresses 0 through 19.
- CPU_4:** CPU_4_IN_PIN is buffered and inverted to produce CPU_4_OUT_PIN signals for addresses 0 through 19.
- CPU_5:** CPU_5_IN_PIN is buffered and inverted to produce CPU_5_OUT_PIN signals for addresses 0 through 19.
- CPU_6:** CPU_6_IN_PIN is buffered and inverted to produce CPU_6_OUT_PIN signals for addresses 0 through 19.
- CPU_7:** CPU_7_IN_PIN is buffered and inverted to produce CPU_7_OUT_PIN signals for addresses 0 through 19.

Multiplexing Data to be Driven onto the CPU Data Bus

This section shows the multiplexing of data to be driven onto the CPU data bus. It includes signals for ExRAM, CPU, and various system components.

- ExRAM Data:** ExRAM_D0_Read{BJ12(a)} through ExRAM_D7_Read{BJ12(h)} are multiplexed into the CPU data bus.
- CPU Data:** CPU_0_OUT_PIN through CPU_7_OUT_PIN are multiplexed into the CPU data bus.
- System Data:** Various system components like \$5206.0_Read_Multiply_Result_D8{BL78}, \$5204.0_Read_Invalid_A0_M2_Sequence{BL52}, \$5205.0_Read_Multiply_Result_D0{BL70}, \$5015.0_Read_Pulse_1_Status_Bit{BL21}, \$5206.1_Read_Multiply_Result_D9{BL77}, \$5205.1_Read_Multiply_Result_D1{BL69}, \$5015.1_Read_Pulse_2_Status_Bit{BL20}, \$5206.2_Read_Multiply_Result_D10{BL76}, \$5205.2_Read_Multiply_Result_D2{BL68}, \$5206.3_Read_Multiply_Result_D11{BL75}, \$5205.3_Read_Multiply_Result_D3{BL67}, \$5206.4_Read_Multiply_Result_D12{BL74}, \$5205.4_Read_Multiply_Result_D4{BL66}, \$5206.5_Read_Multiply_Result_D13{BL73}, \$5205.5_Read_Multiply_Result_D5{BL65}, \$5206.6_Read_Multiply_Result_D14{BL64}, \$5205.6_Read_Multiply_Result_D6{BL72}, \$5010.6_Read_In_Frame{BL80}, \$5206.7_Read_Multiply_Result_D15{BL63}, \$5205.7_Read_Multiply_Result_D7{BL71}, \$5204.7_Read_Scanline_IRQ_Pending{BL79}, and \$5010.7_Read_PCM_IRQ_Pending{BL62} are multiplexed into the CPU data bus.

Decides When to Drive the CPU Data Bus

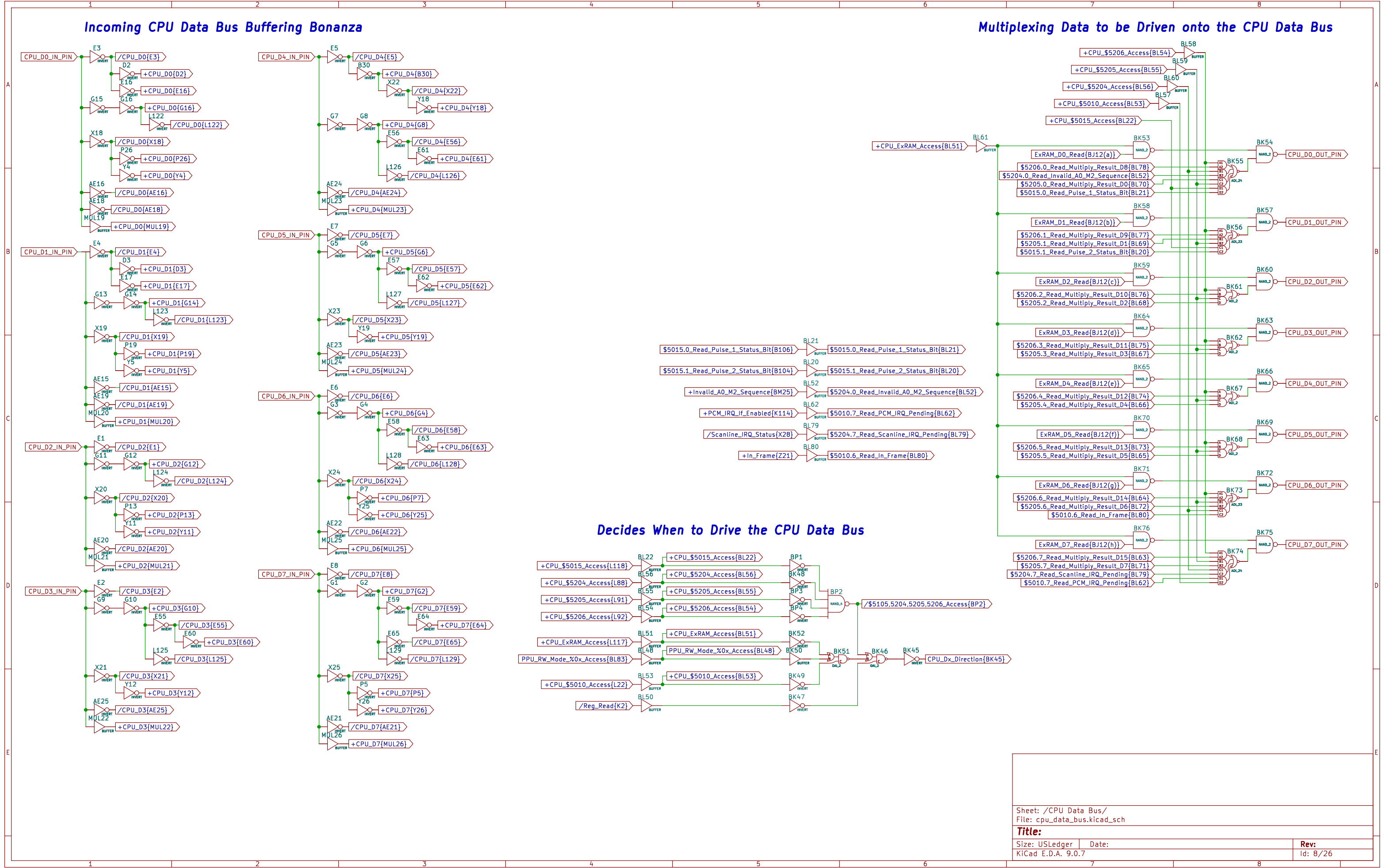
This section shows the logic that decides when to drive the CPU data bus. It includes signals for CPU, ExRAM, and various system components.

- CPU Access:** +CPU_\$5015_Access{BL22}, +CPU_\$5204_Access{BL56}, +CPU_\$5205_Access{BL55}, +CPU_\$5206_Access{BL54}, +CPU_ExRAM_Access{BL51}, +CPU_\$5010_Access{BL53}, and /Reg_Read{K2} are buffered and inverted to produce CPU data bus signals.
- System Data:** Various system components like \$5015.0_Read_Pulse_1_Status_Bit{B106}, \$5015.0_Read_Pulse_1_Status_Bit{BL21}, \$5015.1_Read_Pulse_2_Status_Bit{B104}, \$5015.1_Read_Pulse_2_Status_Bit{BL20}, +Invalid_A0_M2_Sequence{BM25}, \$5204.0_Read_Invalid_A0_M2_Sequence{BL52}, +PCM_IRQ_If_Enabled{K114}, \$5010.7_Read_PCM_IRQ_Pending{BL62}, /Scanline_IRQ_Status{X28}, \$5204.7_Read_Scanline_IRQ_Pending{BL79}, and +In_Frame{Z21} are buffered and inverted to produce CPU data bus signals.

Sheet: /CPU Data Bus/
File: cpu_data_bus.kicad_sch

Title:

Size: USLedger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 8/26



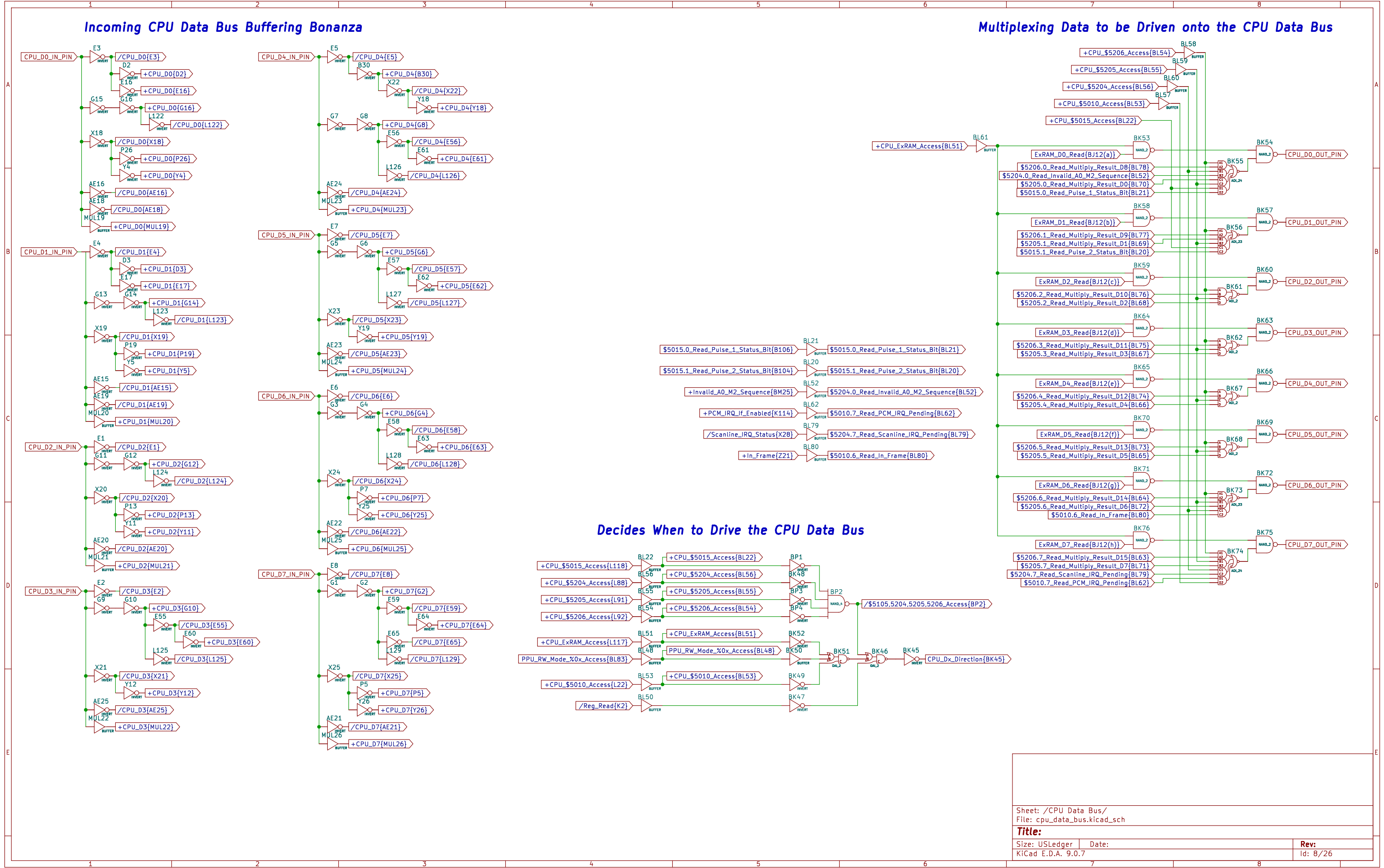
Incoming CPU Data Bus Buffering Bonanza

Decides When to Drive the CPU Data Bus

Multiplexing Data to be Driven onto the CPU Data Bus

Legend:

- AND: AND gate
- OR: OR gate
- XOR: XOR gate
- NOT: NOT gate
- BUF: Buffer
- MUX: Multiplexer
- SEL: Select
- EN: Enable
- DIS: Disable
- IN: Input
- OUT: Output
- DIR: Direction
- MODE: Mode
- DATA: Data
- ADDR: Address
- STATUS: Status
- IRQ: Interrupt Request
- PC: Program Counter
- SP: Stack Pointer
- BP: Base Pointer
- IP: Instruction Pointer
- CR: Control Register
- DR: Data Register
- SR: Status Register
- CR0: Control Register 0
- CR2: Control Register 2
- CR3: Control Register 3
- CR4: Control Register 4
- CR8: Control Register 8
- CR9: Control Register 9
- CR10: Control Register 10
- CR11: Control Register 11
- CR12: Control Register 12
- CR13: Control Register 13
- CR14: Control Register 14
- CR15: Control Register 15
- CR16: Control Register 16
- CR17: Control Register 17
- CR18: Control Register 18
- CR19: Control Register 19
- CR20: Control Register 20
- CR21: Control Register 21
- CR22: Control Register 22
- CR23: Control Register 23
- CR24: Control Register 24
- CR25: Control Register 25
- CR26: Control Register 26
- CR27: Control Register 27
- CR28: Control Register 28
- CR29: Control Register 29
- CR30: Control Register 30
- CR31: Control Register 31
- CR32: Control Register 32
- CR33: Control Register 33
- CR34: Control Register 34
- CR35: Control Register 35
- CR36: Control Register 36
- CR37: Control Register 37
- CR38: Control Register 38
- CR39: Control Register 39
- CR40: Control Register 40
- CR41: Control Register 41
- CR42: Control Register 42
- CR43: Control Register 43
- CR44: Control Register 44
- CR45: Control Register 45
- CR46: Control Register 46
- CR47: Control Register 47
- CR48: Control Register 48
- CR49: Control Register 49
- CR50: Control Register 50
- CR51: Control Register 51
- CR52: Control Register 52
- CR53: Control Register 53
- CR54: Control Register 54
- CR55: Control Register 55
- CR56: Control Register 56
- CR57: Control Register 57
- CR58: Control Register 58
- CR59: Control Register 59
- CR60: Control Register 60
- CR61: Control Register 61
- CR62: Control Register 62
- CR63: Control Register 63
- CR64: Control Register 64
- CR65: Control Register 65
- CR66: Control Register 66
- CR67: Control Register 67
- CR68: Control Register 68
- CR69: Control Register 69
- CR70: Control Register 70
- CR71: Control Register 71
- CR72: Control Register 72
- CR73: Control Register 73
- CR74: Control Register 74
- CR75: Control Register 75
- CR76: Control Register 76
- CR77: Control Register 77
- CR78: Control Register 78
- CR79: Control Register 79
- CR80: Control Register 80
- CR81: Control Register 81
- CR82: Control Register 82
- CR83: Control Register 83
- CR84: Control Register 84
- CR85: Control Register 85
- CR86: Control Register 86
- CR87: Control Register 87
- CR88: Control Register 88
- CR89: Control Register 89
- CR90: Control Register 90
- CR91: Control Register 91
- CR92: Control Register 92
- CR93: Control Register 93
- CR94: Control Register 94
- CR95: Control Register 95
- CR96: Control Register 96
- CR97: Control Register 97
- CR98: Control Register 98
- CR99: Control Register 99
- CR100: Control Register 100
- CR101: Control Register 101
- CR102: Control Register 102
- CR103: Control Register 103
- CR104: Control Register 104
- CR105: Control Register 105
- CR106: Control Register 106
- CR107: Control Register 107
- CR108: Control Register 108
- CR109: Control Register 109
- CR110: Control Register 110
- CR111: Control Register 111
- CR112: Control Register 112
- CR113: Control Register 113
- CR114: Control Register 114
- CR115: Control Register 115
- CR116: Control Register 116
- CR117: Control Register 117
- CR118: Control Register 118
- CR119: Control Register 119
- CR120: Control Register 120
- CR121: Control Register 121
- CR122: Control Register 122
- CR123: Control Register 123
- CR124: Control Register 124
- CR125: Control Register 125
- CR126: Control Register 126
- CR127: Control Register 127
- CR128: Control Register 128
- CR129: Control Register 129
- CR130: Control Register 130
- CR131: Control Register 131
- CR132: Control Register 132
- CR133: Control Register 133
- CR134: Control Register 134
- CR135: Control Register 135
- CR136: Control Register 136
- CR137: Control Register 137
- CR138: Control Register 138
- CR139: Control Register 139
- CR140: Control Register 140
- CR141: Control Register 141
- CR142: Control Register 142
- CR143: Control Register 143
- CR144: Control Register 144
- CR145: Control Register 145
- CR146: Control Register 146
- CR147: Control Register 147
- CR148: Control Register 148
- CR149: Control Register 149
- CR150: Control Register 150
- CR151: Control Register 151
- CR152: Control Register 152
- CR153: Control Register 153
- CR154: Control Register 154
- CR155: Control Register 155
- CR156: Control Register 156
- CR157: Control Register 157
- CR158: Control Register 158
- CR159: Control Register 159
- CR160: Control Register 160
- CR161: Control Register 161
- CR162: Control Register 162
- CR163: Control Register 163
- CR164: Control Register 164
- CR165: Control Register 165
- CR166: Control Register 166
- CR167: Control Register 167
- CR168: Control Register 168
- CR169: Control Register 169
- CR170: Control Register 170
- CR171: Control Register 171
- CR172: Control Register 172
- CR173: Control Register 173
- CR174: Control Register 174
- CR175: Control Register 175
- CR176: Control Register 176
- CR177: Control Register 177
- CR178: Control Register 178
- CR179: Control Register 179
- CR180: Control Register 180
- CR181: Control Register 181
- CR182: Control Register 182
- CR183: Control Register 183
- CR184: Control Register 184
- CR185: Control Register 185
- CR186: Control Register 186
- CR187: Control Register 187
- CR188: Control Register 188
- CR189: Control Register 189
- CR190: Control Register 190
- CR191: Control Register 191
- CR192: Control Register 192
- CR193: Control Register 193
- CR194: Control Register 194
- CR195: Control Register 195
- CR196: Control Register 196
- CR197: Control Register 197
- CR198: Control Register 198
- CR199: Control Register 199
- CR200: Control Register 200
- CR201: Control Register 201
- CR202: Control Register 202
- CR203: Control Register 203
- CR204: Control Register 204
- CR205: Control Register 205
- CR206: Control Register 206
- CR207: Control Register 207
- CR208: Control Register 208
- CR209: Control Register 209
- CR210: Control Register 210
- CR211: Control Register 211
- CR212: Control Register 212
- CR213: Control Register 213
- CR214: Control Register 214
- CR215: Control Register 215
- CR216: Control Register 216
- CR217: Control Register 217
- CR218: Control Register 218
- CR219: Control Register 219
- CR220: Control Register 220
- CR221: Control Register 221
- CR222: Control Register 222
- CR223: Control Register 223
- CR224: Control Register 224
- CR225: Control Register 225
- CR226: Control Register 226
- CR227: Control Register 227
- CR228: Control Register 228
- CR229: Control Register 229
- CR230: Control Register 230
-



Title:		
Size: USLedger	Date:	Rev:
KiCad E.D.A. 9.0.7		Id: 8/26

\$5010 Register Mode & IRQ Enable

The schematic illustrates the internal logic of the \$5010 Register Mode & IRQ Enable circuit. It shows the data bus connections, read/write control logic, and the DAC outputs. The circuit is controlled by signals from the CPU (G16, G14, G12, G10, G8, G6, G4, G2) and the \$5010 register (L23, L26, L24). The DAC outputs are labeled PCM_DAC_Bit_0 through PCM_DAC_Bit_7. The IRQ enable output is labeled +PCM_IRQ_If_Enabled(K114).

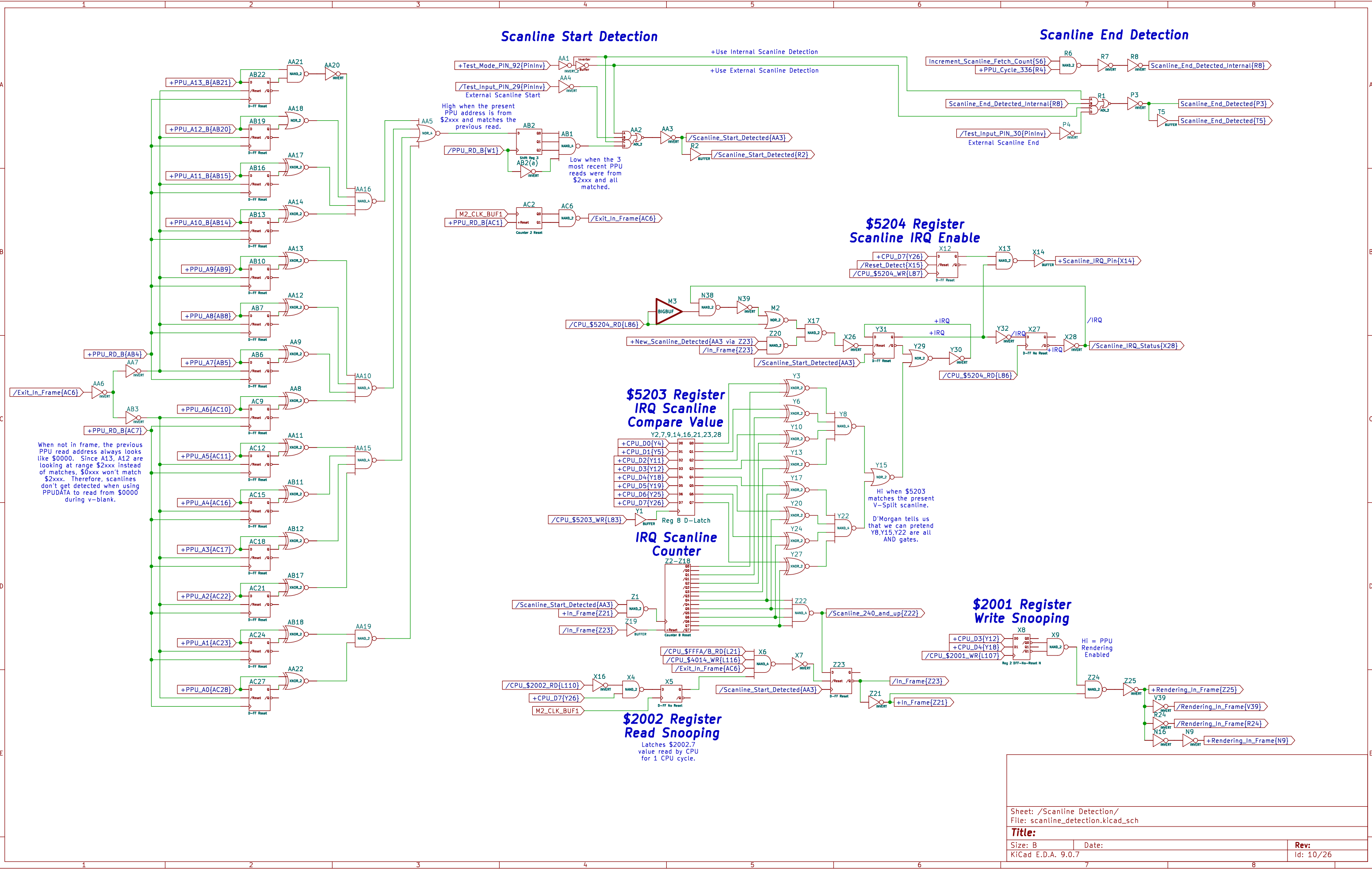
Writing to \$5011 in write mode:

Reading from \$8000-BFFF in read mode:

Notes:

- The differential clockbursts' outputs are flipped in these tribufs, which makes it have +OE instead of /OE.
- When the value written is \$00, this loops back the existing value instead of accepting the value \$00.

Title:		
Size: USLedger	Date:	Rev:
KiCad E.D.A. 9.0.7		Id: 9/26



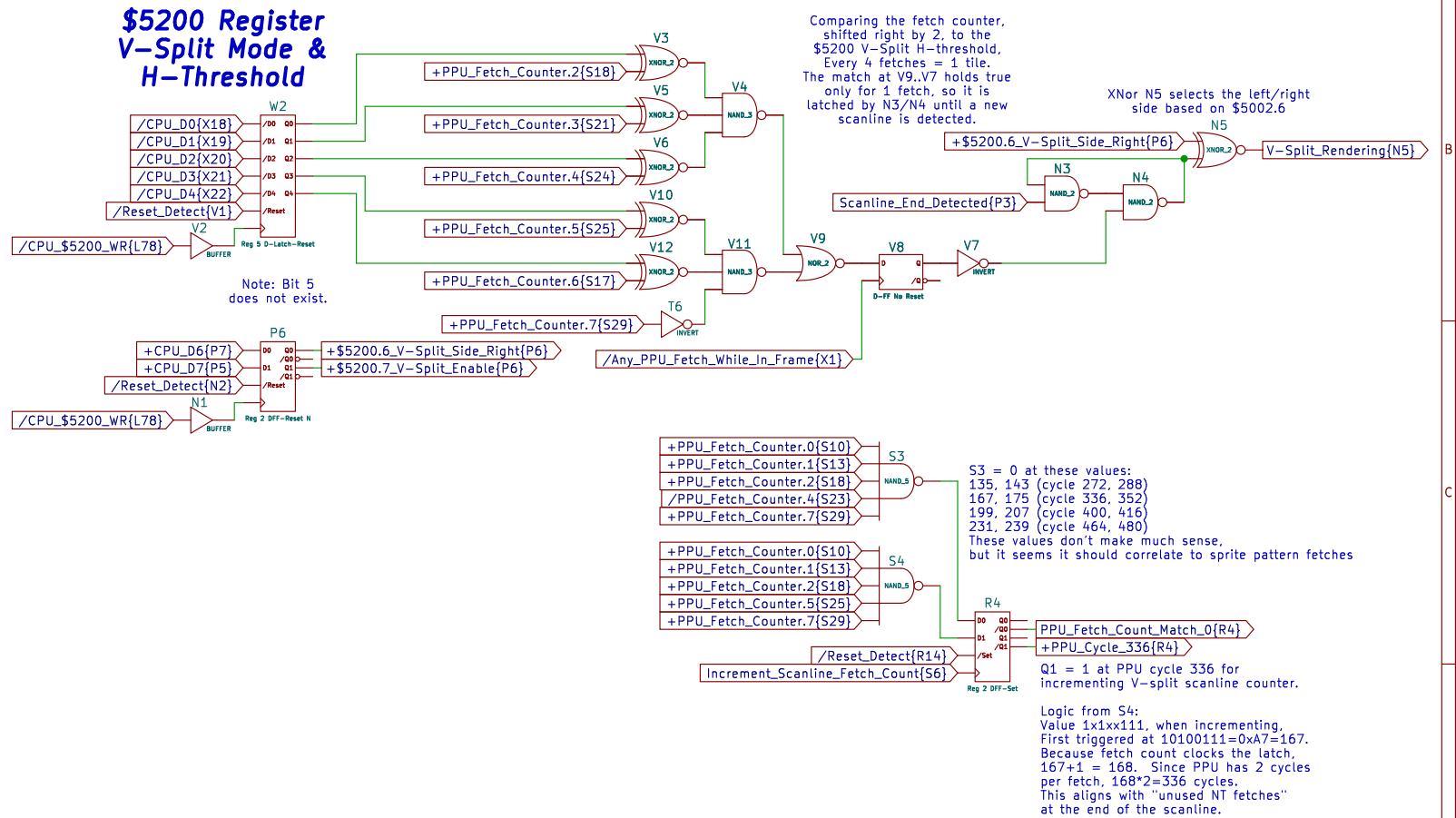
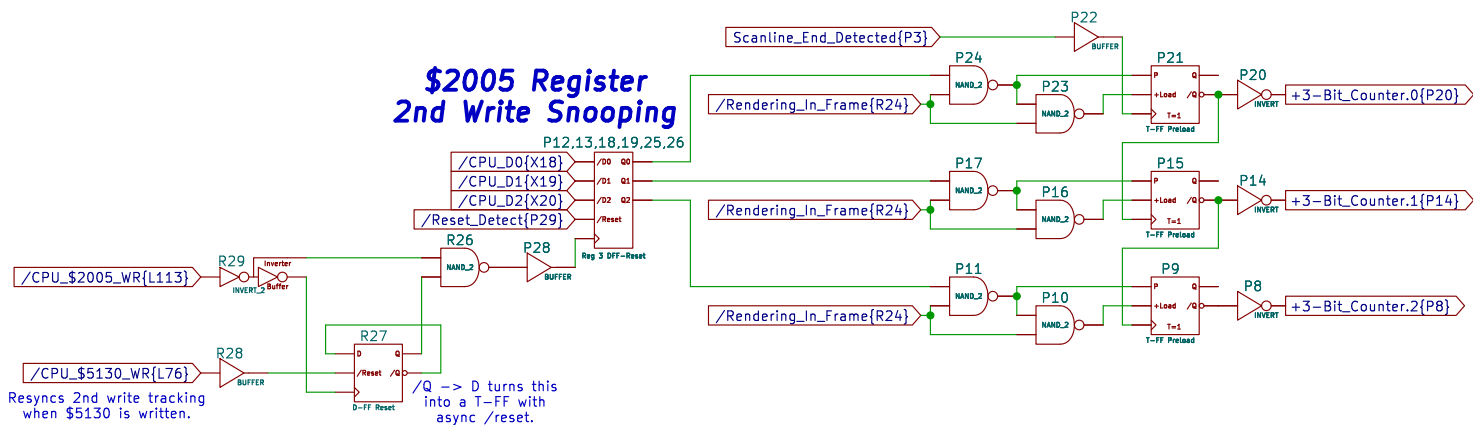
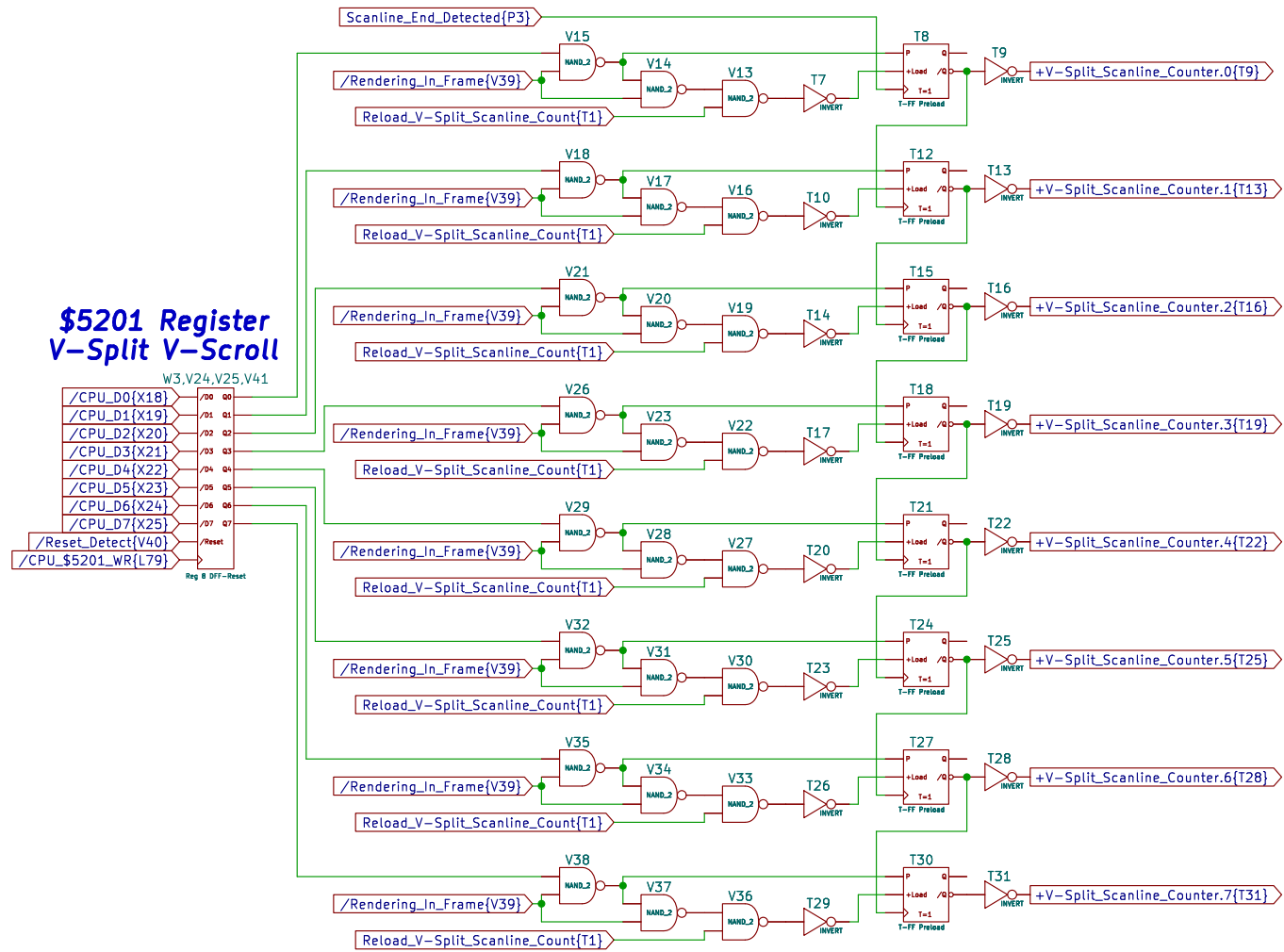
When not in frame, the previous PPU read address always looks like \$0000. Since A13, A12 are looking at range \$2xxx instead of matches, \$0xxx won't match \$2xxx. Therefore, scanlines don't get detected when using PPU_DATA to read from \$0000 during v-blank.

High when the present PPU address is from \$2xxx and matches the previous read.

Low when the 3 most recent PPU reads were from \$2xxx and all matched.

Hi when \$5203 matches the present V-Split scanline.

D'Morgan tells us that we can pretend Y8, Y15, Y22 are all AND gates.



Size: B	Date:	Rev:
KiCad E.D.A. 9.0.7		Id: 11/26

1	2	3	4	5	6	7	8	
---	---	---	---	---	---	---	---	--



The diagram illustrates the 10x2x1 Multiplexer, which has 10 inputs and 10 outputs. The inputs are labeled M1 through M10, and the outputs are labeled M1 through M10. The inputs are connected to the outputs via a series of multiplexers. The inputs are:

- +PPU_Fetch_Counter.0{S10}
- +PPU_Fetch_Counter.2{S18}
- +PPU_Fetch_Counter.4{S24}
- +PPU_Fetch_Counter.3{S21}
- +PPU_Fetch_Counter.5{S25}
- +PPU_Fetch_Counter.4{S24}
- +PPU_Fetch_Counter.6{S17}
- +PPU_Fetch_Counter.5{S25}
- +V-Split_Scanline_Counter.5{T25}
- +PPU_Fetch_Counter.6{S17}
- +V-Split_Scanline_Counter.6{T28}
- +V-Split_Scanline_Counter.3{T19}
- +V-Split_Scanline_Counter.7{T31}
- +V-Split_Scanline_Counter.4{T22}
- +V-Split_Scanline_Counter.5{T25}
- +V-Split_Scanline_Counter.6{T28}
- +V-Split_Scanline_Counter.7{T31}

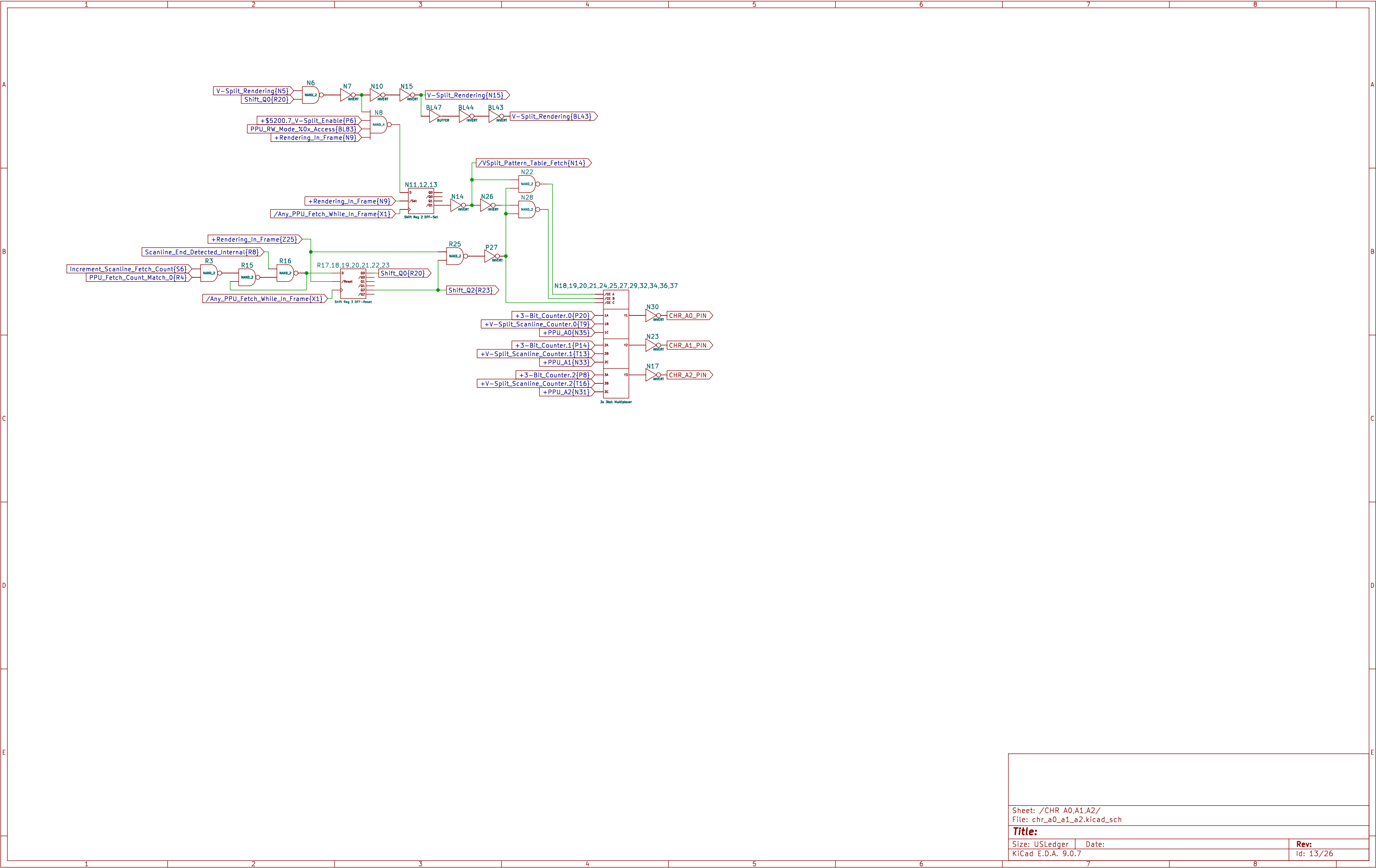
The outputs are labeled M1 through M10, and the inputs are connected to the outputs via a series of multiplexers. The outputs are:

- PPU_Fetch_Mux_0{M1}
- PPU_Fetch_Mux_1{M1}
- PPU_Fetch_Mux_2{M1}
- PPU_Fetch_Mux_3{M1}
- PPU_Fetch_Mux_4{M1}
- PPU_Fetch_Mux_5{M1}
- PPU_Fetch_Mux_6{M1}
- PPU_Fetch_Mux_7{M1}
- PPU_Fetch_Mux_8{M1}
- PPU_Fetch_Mux_9{M1}

The diagram shows the internal structure of the multiplexer, including the 10x2x1 Multiplexer block and the 10x2x1 Multiplexer block.



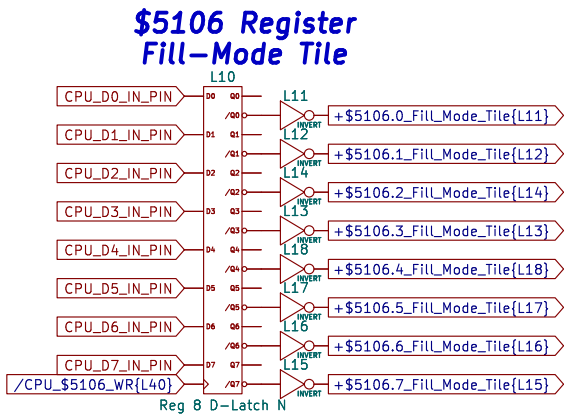
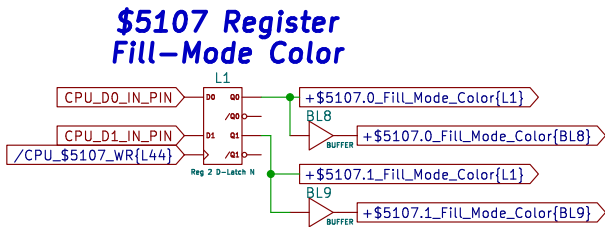
Counter.6: unclear if + or /



\$5104 Register ExRAM Mode

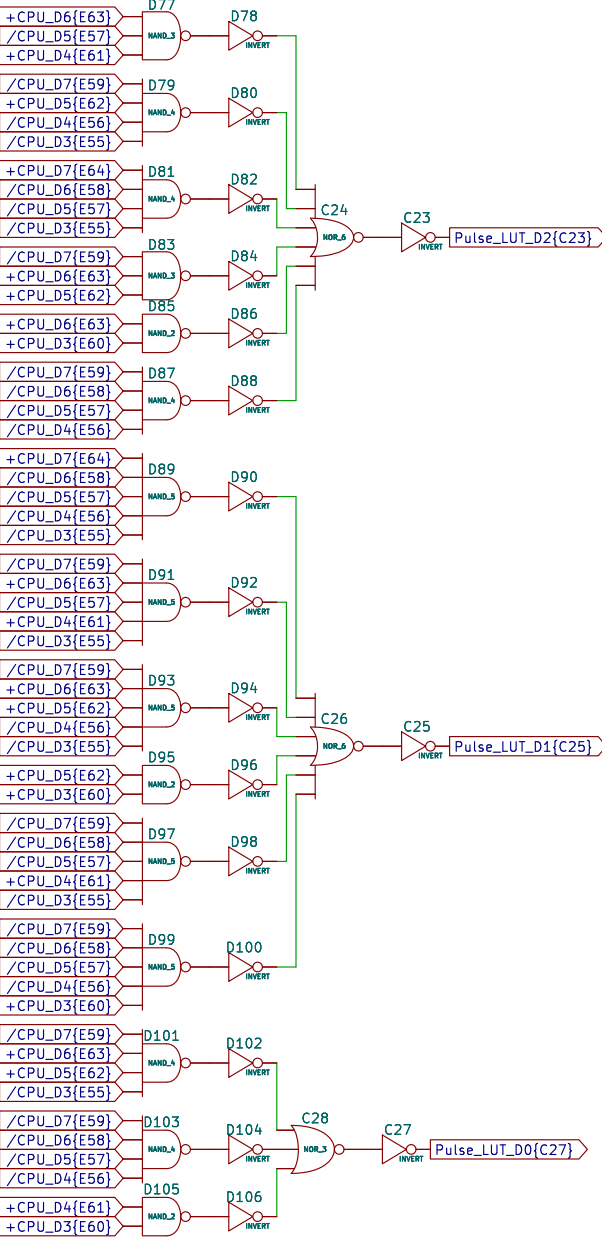
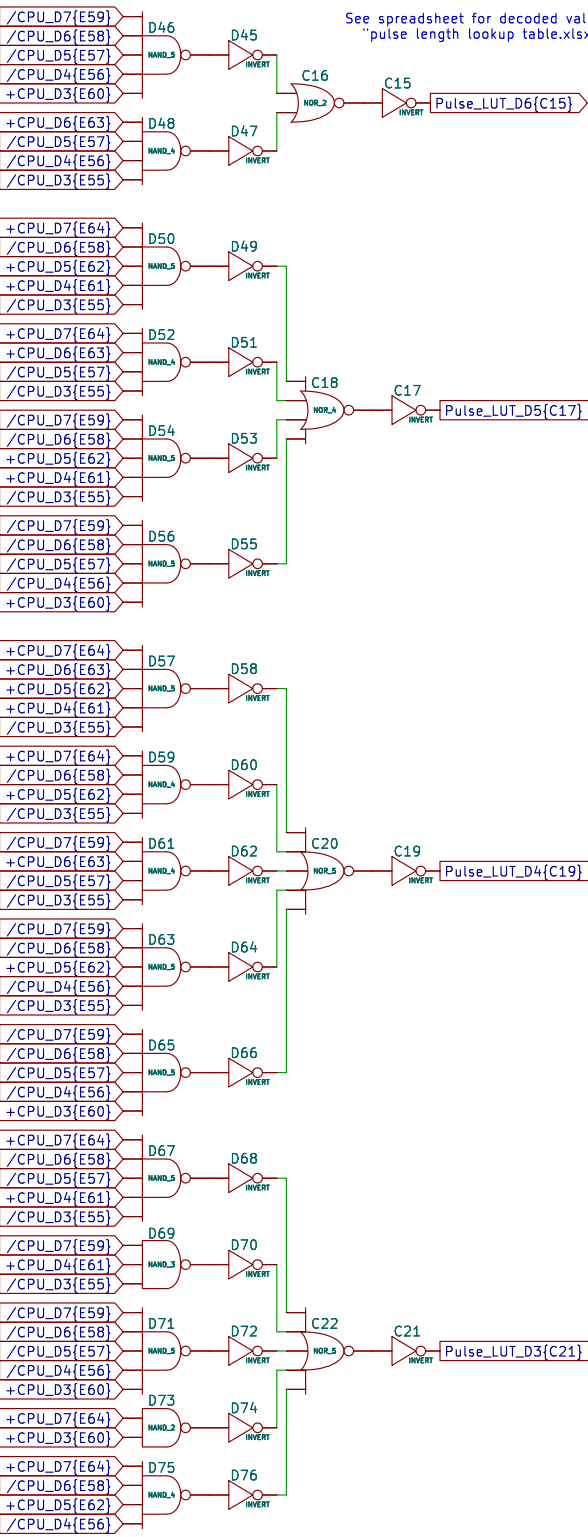
Signal A9 (+ExRAM Nametable Access)
is VERY closeby but no obvious connection

Rev: 14/26



5-Bit to 7-Bit Pulse Length Lookup Table

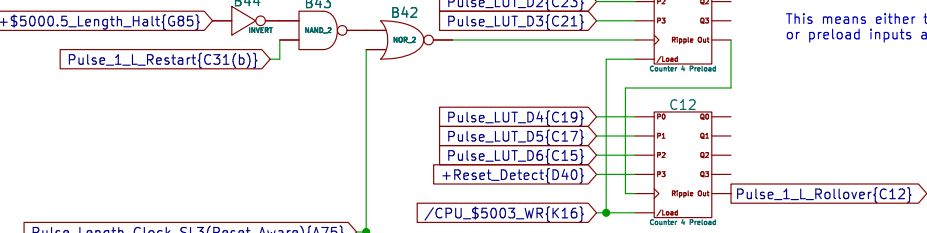
See spreadsheet for decoded values:
"pulse length lookup table.xlsx"



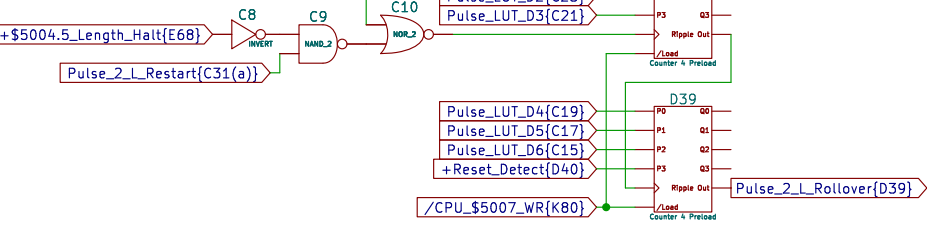
\$5003 Register Pulse 1 Length Counter

These counters count "down"
the preloaded period value.

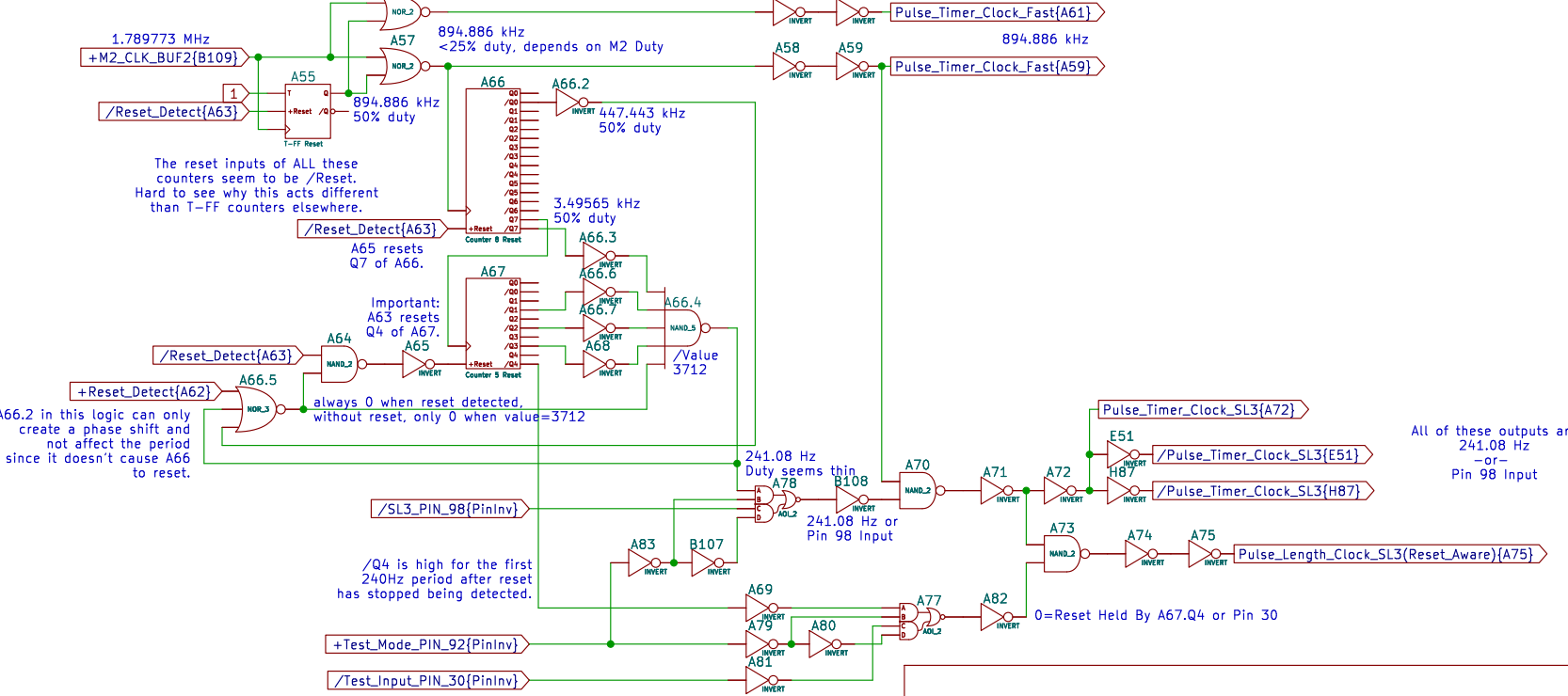
This means either the outputs
or preload inputs are inverted.

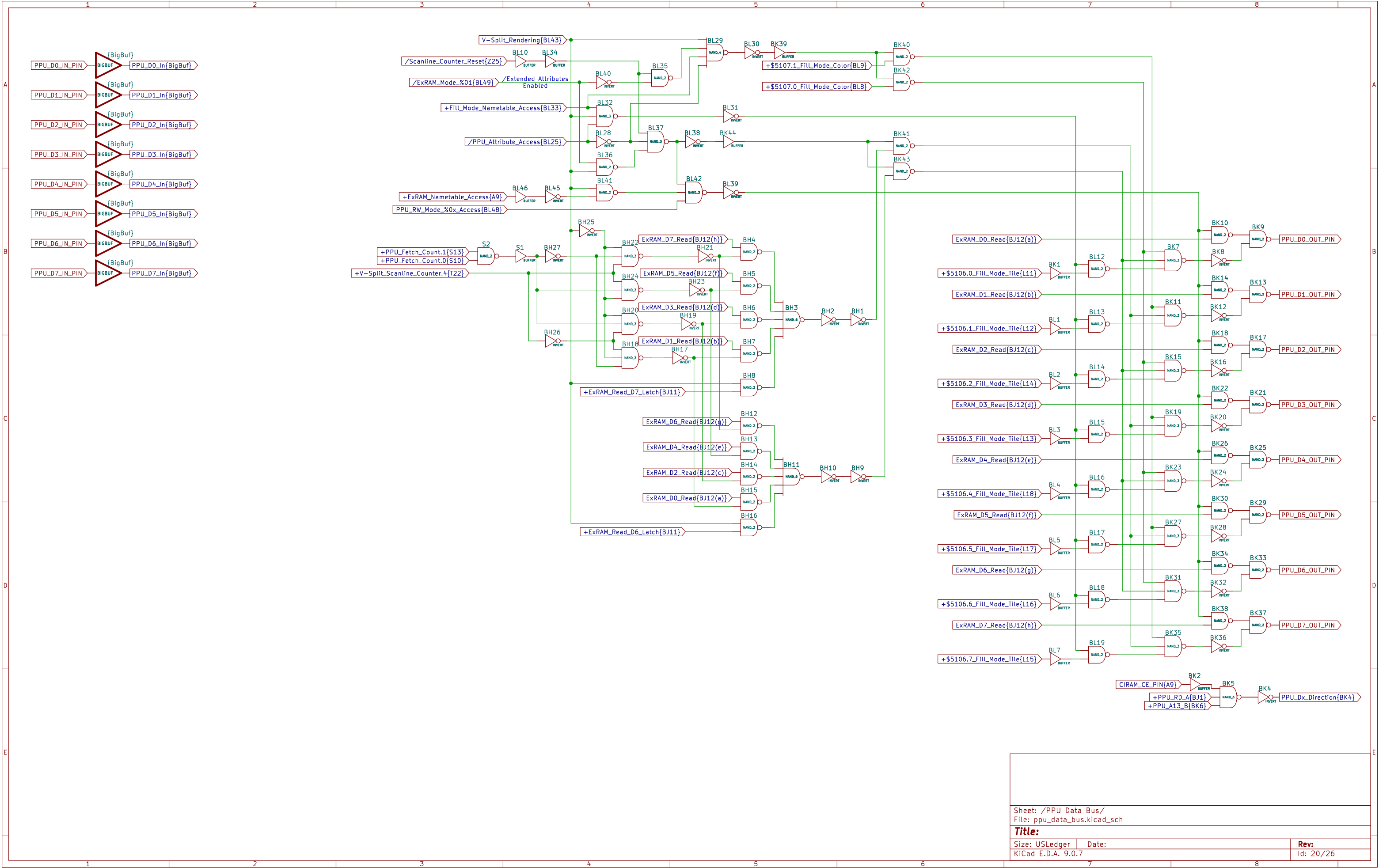


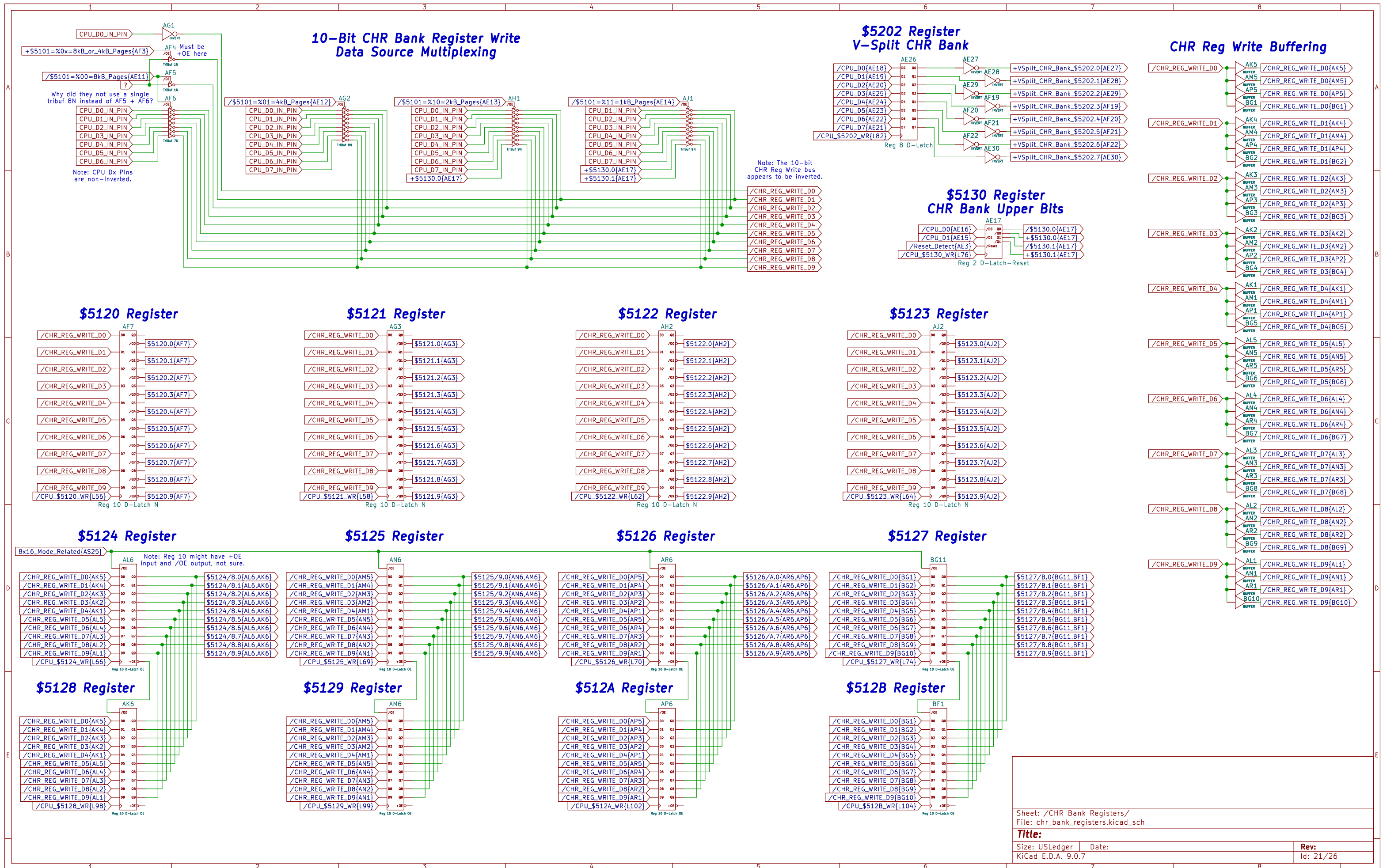
\$5007 Register Pulse 2 Length Counter

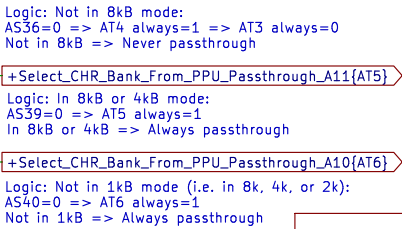
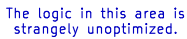
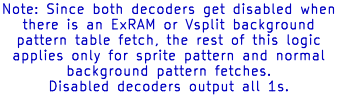


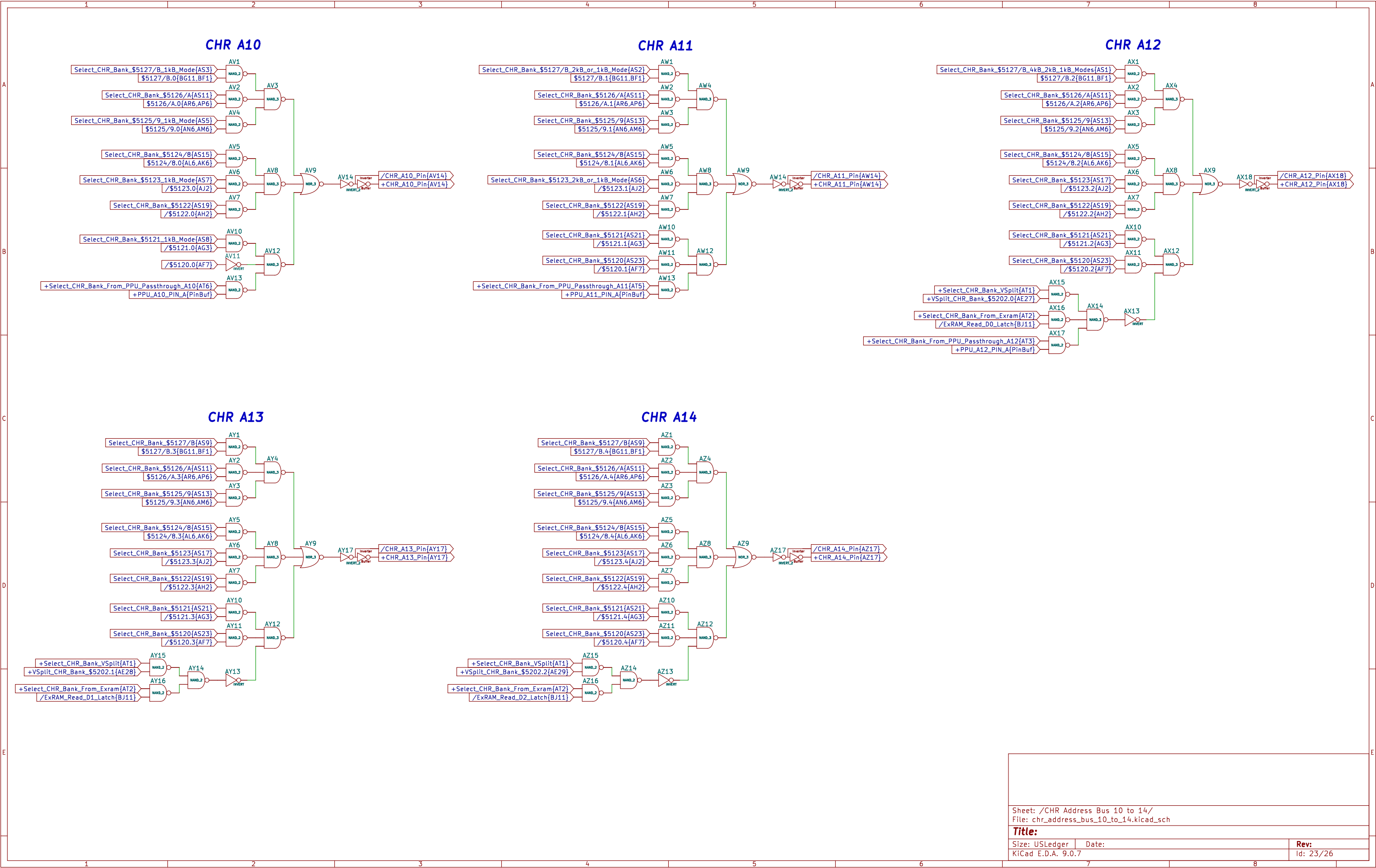
Pulse Channel Clock Source

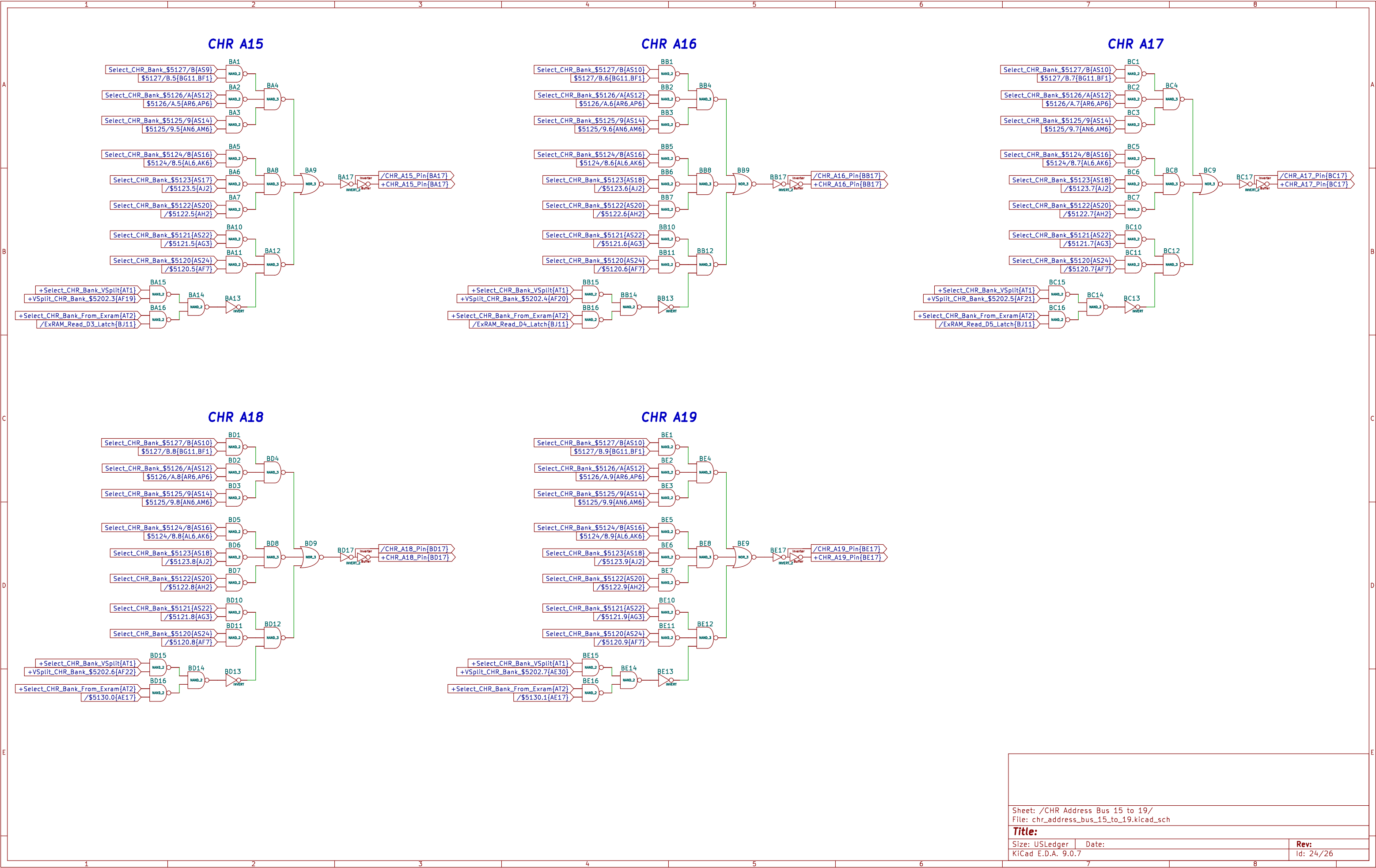










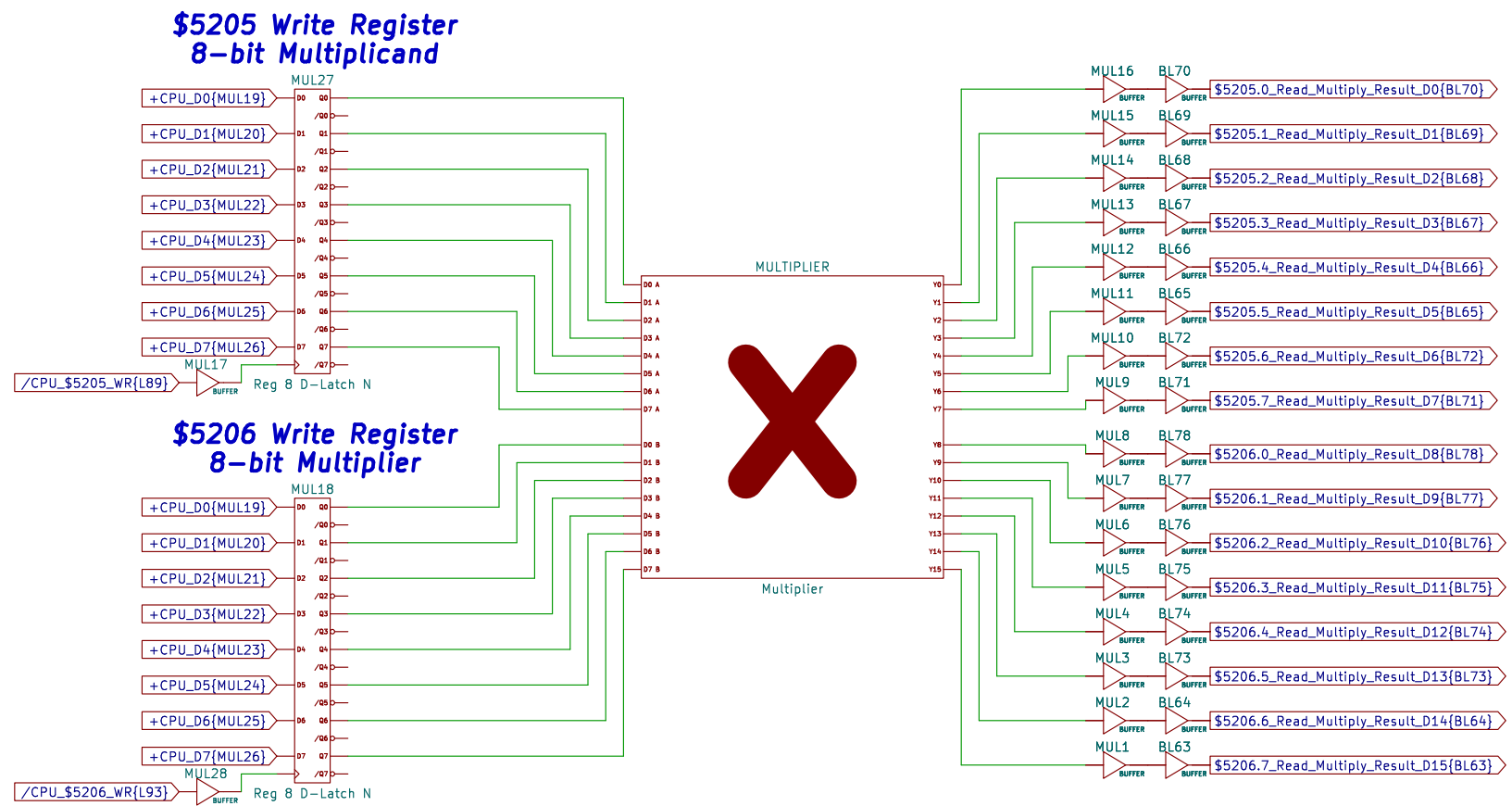


Sheet: /CHR Address Bus 15 to 19/
File: chr_address_bus_15_to_19.kicad_sch

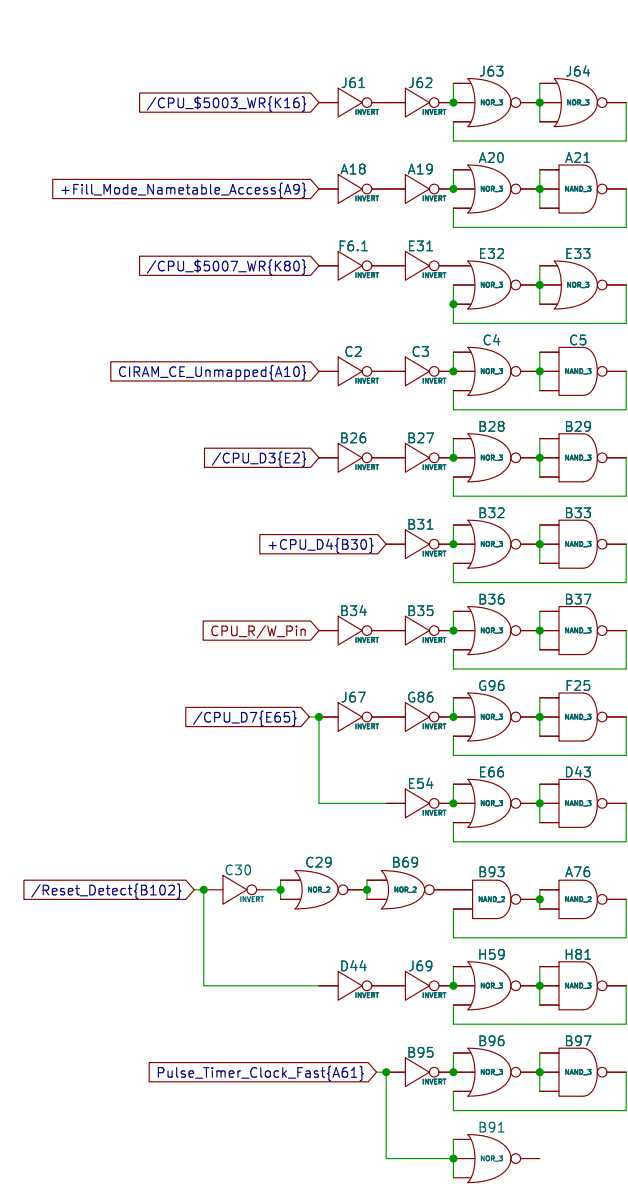
Title:

Size: USLdger Date:
KiCad E.D.A. 9.0.7

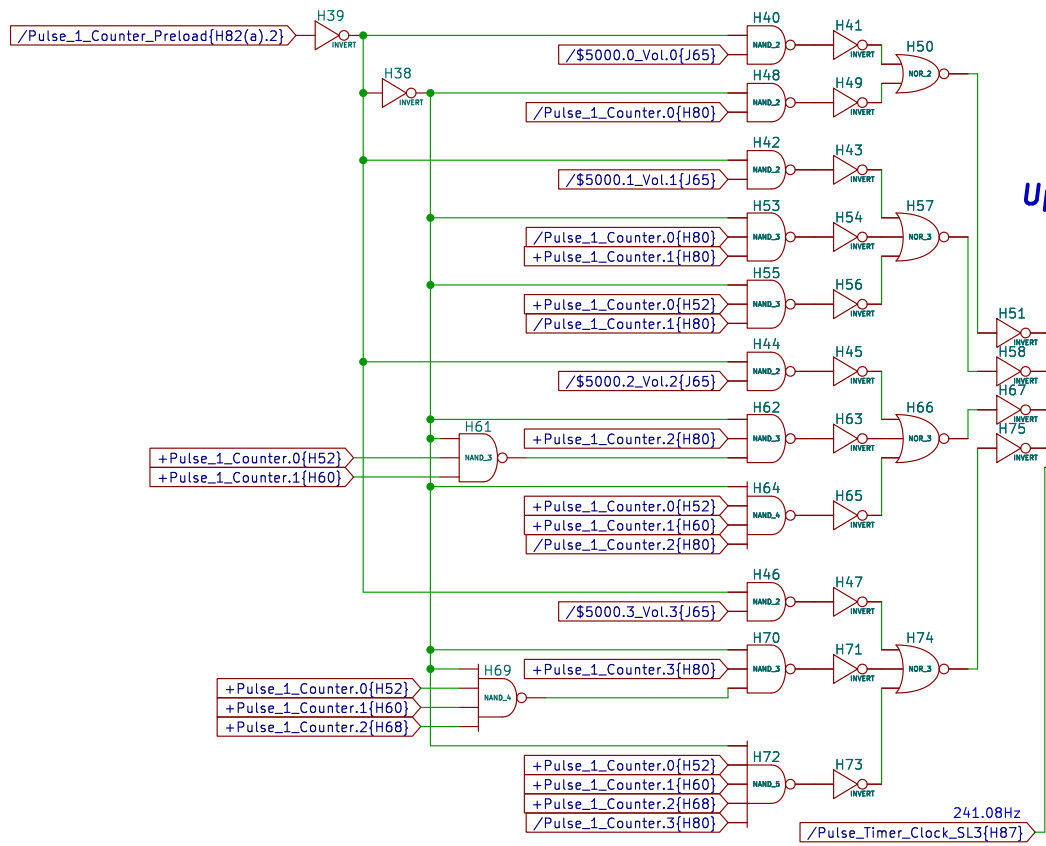
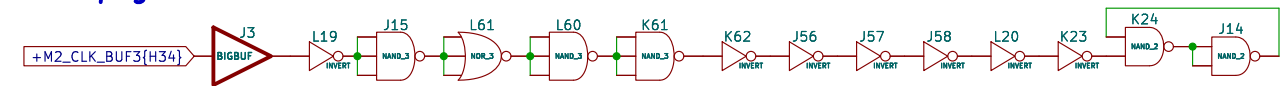
Rev:
Id: 24/26



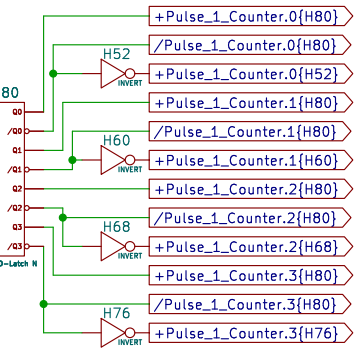
This sheet contains all residual items that don't serve any apparent purpose.



M2 Spaghetti That Goes Nowhere

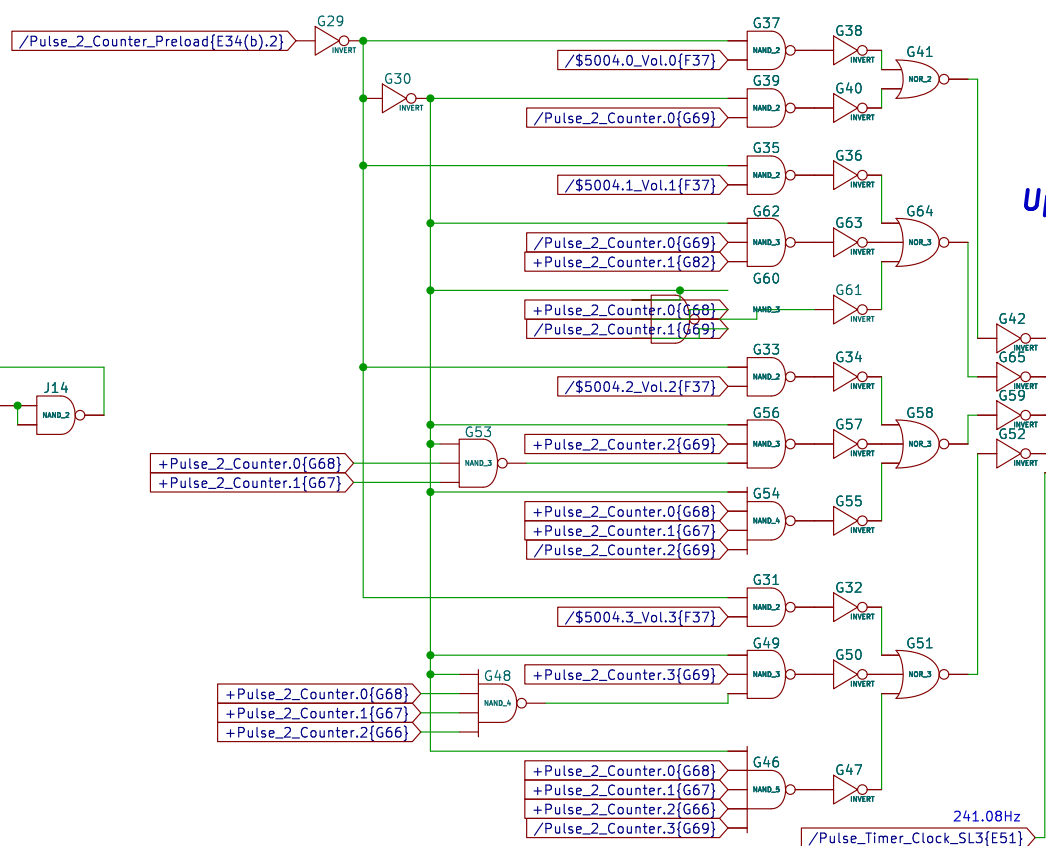


Pulse 1
Unused 4-bit Binary
Up-Counter With Preload

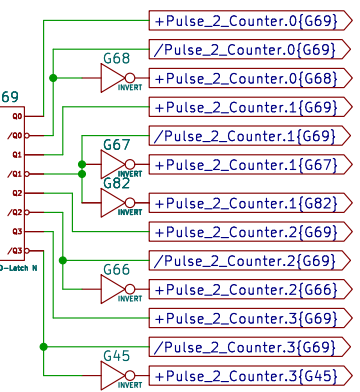


The only output that goes elsewhere is H78, which gets latched by H82(a), coming directly back here as /Pulse_1_Counter_Preload; all self-contained. It does not appear to provide any other external signals, so I don't think it actually does anything. It seems like an abandoned envelope counter that was left in.

It latches the 4-bit pulse volume, then counts up at a rate of 240 Hz. Nothing stops it from counting or rolling over from \$F back to \$0.



Pulse 2
Unused 4-bit Binary
Up-Counter With Preload



The only output that goes elsewhere is H78, which gets latched by H82(a), coming directly back here as /Pulse_1_Counter_Preload; all self-contained. It does not appear to provide any other external signals, so I don't think it actually does anything. It seems like an abandoned envelope counter that was left in.

It latches the 4-bit pulse volume, then counts up at a rate of 240 Hz. Nothing stops it from counting or rolling over from \$F back to \$0.